Command-line commands

GitHub Copilot CLI command reference

Find commands and keyboard shortcuts to help you use Copilot CLI effectively.

Copy markdown

Command-line commands

`copilot`	Launch the interactive user interface.
`copilot app`	Open the GitHub Copilot app in the current directory, deep-linking straight to a new session. Falls back to opening the app's download page in a browser if the deep link can't be handled, or prints the URL to open manually if no browser is available.
`copilot completion SHELL`	Print a shell script for the chosen shell that can be used to enable tab completion for Copilot CLI. Supported shells: `bash`, `zsh`, `fish`. See Using `copilot completion`.
`copilot config [KEY] [VALUE]`	Read or change Copilot settings from the command line, outside an interactive session. See Using `copilot config`.
`copilot help [TOPIC]`	Display help information. Help topics include: `billing`, `config`, `commands`, `environment`, `logging`, `monitoring`, `permissions`, `providers`, and `sandbox`.
`copilot init`	Initialize Copilot custom instructions for this repository.
`copilot login [OPTION]`	Authenticate with Copilot via OAuth. See `copilot login` options.

`copilot mcp`	Manage MCP server configurations from the command line.
`copilot memories import FILE`	Import prepared memories from a JSONL file. See Importing memories.
`copilot plugin`	Manage plugins and plugin marketplaces, including listing, enabling, disabling, and uninstalling them. `copilot plugins` (plural) is a legacy alias. See GitHub Copilot CLI plugin reference.
`copilot instruction`	Non-interactively list custom instruction sources discovered for the current working directory. See Using `copilot instruction`.
`copilot lsp`	Non-interactively list configured language servers. See Using `copilot lsp`.
`copilot sandbox ca`	Manage the sandbox's proxy certificate authority from outside an interactive session. See Using `copilot sandbox ca`.
`copilot skill`	Manage agent skills from the command line (list, add, remove, enable, and disable skills). See Managing skills non-interactively.
`copilot update`	Download and install the latest version. To use a specific version, see "Pinning the CLI version" in GitHub Copilot CLI programmatic reference.
`copilot version`	Display version information and check for updates.
`copilot workflow run WORKFLOW-NAME`	Run an existing dynamic workflow directly, without sending a chat prompt. Waits for the run to finish or stop. See Using `copilot workflow run`.

Command	Purpose
`--host HOST`	GitHub host URL (default: `https://github.com`). Use this to authenticate with a GitHub Enterprise Cloud instance that uses data residency (for example, `https://example.ghe.com`).
`--web-flow`	Force the browser-based (web) authentication flow, which is the default on a local terminal.
`--device-code`	Force the OAuth device code flow, which is the default on a remote terminal or in CI.

Option	Purpose
`--with-token`	Read an authentication token from standard input instead of starting an OAuth flow. Prompts without echoing input when standard input is a terminal.

On a local terminal, or a non-TTY local process that can still reach a browser (for example, a desktop OS, or Linux with a display or browser signal), the default authentication mode is a browser-based web flow: the CLI opens your browser to authorize, then captures the result on a local loopback callback. On a remote terminal (for example, SSH, GitHub Codespaces, or a dev container) or in CI, the CLI defaults to the OAuth device code flow instead, since the browser typically can't reach the loopback port. Use `--web-flow` or `--device-code` to override the automatically selected flow.

After completion, an authentication token is stored securely in the system credential store. If a credential store is not found, the token is stored in a plain text configuration file under `~/.copilot/` (or the directory specified by `COPILOT_HOME` if set).

Alternatively, Copilot CLI will use an authentication token found in environment variables. The following are checked in order of precedence: `COPILOT_GITHUB_TOKEN`, `GH_TOKEN`, `GITHUB_TOKEN`. This method is most suitable for headless use such as automation.

Supported token types include fine-grained personal access tokens (v2 PATs) with the "Copilot Requests" permission, OAuth tokens from the Copilot CLI app, and OAuth tokens from the GitHub CLI (`gh`) app. Classic personal access tokens (`ghp_`) are not supported.

Examples:

```
# Authenticate with github.com
copilot login

# Authenticate with GitHub Enterprise Cloud (data residency)
copilot login --host https://example.ghe.com

# Force the device code flow, for example on a remote terminal
copilot login --device-code

# Force the browser (web) flow, for example on a local terminal
copilot login --web-flow

# Read a token from standard input, bypassing the OAuth flow entirely
copilot login --with-token < mytoken.txt

# Use a fine-grained PAT via environment variable
COPILOT_GITHUB_TOKEN=github_pat_... copilot
```

Using `copilot completion`

The command `copilot completion SHELL` outputs a script for the specified shell (bash, zsh, or fish).

By sourcing this script (or writing it to your shell's completion directory) you can enable tab completion in your terminal for `copilot` subcommands, command options, and known value

choices for command options. For `copilot config`, completion also covers setting keys, boolean and enum values, and files or directories for path-valued settings.

Usage examples

Bash (current session only):

```bash
source <(copilot completion bash)
```

Bash (persistent, Linux):

```bash
copilot completion bash | sudo tee /etc/bash_completion.d/copilot
```

Zsh – write the output to a directory on your $fpath. Restart the shell after running this command:

```shell
copilot completion zsh > "${fpath[1]}/_copilot"
```

Fish:

```shell
copilot completion fish > ~/.config/fish/completions/copilot.fish
```

Managing plugins non-interactively

Use `copilot plugin` to install, list, update, enable, disable, and uninstall plugins from the command line, without opening an interactive session. `copilot plugins` (plural) is a legacy alias for the same command. For the full command and option reference, see GitHub Copilot CLI plugin reference.

Using `copilot config`

Run `copilot config` to read and change Copilot settings from the command line, outside an interactive session. This is the non-interactive counterpart to the `/settings` (or `/config`) slash command. For more information about the settings file and its keys, see GitHub Copilot CLI configuration directory.

```
# Print a setting
copilot config theme
```

```
# Set a nested value (dot notation)
copilot config footer.showQuota off

# Append an item to a list setting (added only if not already present)
copilot config allowedUrls github.com

# Append an item that starts with a dash
copilot config powershellFlags -- -NoProfile

# Remove a key, or one list item
copilot config --rm theme
copilot config --rm allowedUrls github.com

# List settings as key=value lines, or as JSON
copilot config --list
copilot config --repo --list --json
```

Option/Argument	Description
`KEY`	Setting key in dot notation (for example, `footer.showQuota`).
`VALUE`	Value to set, append, or remove from a list with `--rm`.
`--list`	List settings as `key=value` lines; conflicts with `KEY` / `VALUE` / `--rm`.
`--json`	Output `--list` as JSON; requires `--list`.
`--rm`	Remove the key, or only the given item from a list; requires `KEY`
`--global`	Use your user settings file (the default); conflicts with `--repo` / `--local`.
`--repo`	Use the repository's `.github/copilot/settings.json`; conflicts with `--global` / `--local`.
`--local`	Use the repository's `.github/copilot/settings.local.json`; conflicts with `--global` / `--repo`.

Values must match the setting's type: `on` / `off` or `true` / `false` for booleans, a number for numeric settings, or an accepted choice for enums. Reading a list prints one item per line; reading a group prints JSON. An unset key exits with status `1`. Secret values are redacted from output.

Commands use your user settings file (`settings.json` in `~/.copilot`, or in `$COPILOT_HOME` when set) by default. `--repo` and `--local` accept only the settings a repository can override—see GitHub Copilot CLI configuration directory. Run `copilot help config` for the available settings.

Using `copilot instruction`

Run `copilot instruction list` to non-interactively list custom instruction sources discovered for the current working directory. This replaces the retired `copilot plugins list --kind instruction`.

```
copilot instruction list
copilot instruction list --json
```

Option	Description
`--json`	Emit JSON instead of text.
`--config-dir=DIRECTORY`	Path to the configuration directory. This option is deprecated. Use `COPILOT_HOME` instead.

Each `--json` entry has the shape `{ id, label, description?, location, type, sourcePath, defaultDisabled, applyTo? }`. This listing resolves plugin-contributed instructions against your global user settings only—an instruction enabled by a trusted repository

GitHub Docs Select your plan: Free, Pro, & Search or ask Copilot / Menu English Sign up

Run `copilot lsp list` to non-interactively list configured language servers. This replaces the retired `copilot plugins list --kind lsp`. This command is inspect-only; use the `/lsp` slash command in an interactive session to start, stop, or reload a language server.

```
copilot lsp list
copilot lsp list --json
```

Option	Description
`--json`	Emit JSON instead of text.
`--config-dir=DIRECTORY`	Path to the configuration directory. This option is deprecated. Use `COPILOT_HOME` instead.

Each `--json` entry has the shape `{ id, fileExtensions?, sourcePlugin? }`.

Custom agents and session-scoped hooks aren't covered by `copilot instruction`, `copilot lsp`, `copilot plugin`, `copilot mcp`, or `copilot skill`. All require a live session.

Importing memories

Use `copilot memories import` to validate and import user-level preferences and repository-level facts from a JSONL file.

```
copilot memories import PATH-TO-FILE
```

Option	Description
`--dry-run`	Validate the complete file without authenticating or submitting memories.
`--on-conflict skip|error`	Skip conflicting records or treat them as errors. The default is `error`.
`--output json`	Return the import result as JSON instead of human-readable text.

If any line fails schema validation, no records are submitted. After validation succeeds, records are submitted individually, so a service failure can result in a partial import. Check the per-record results and retry only failed records. The `--on-conflict skip` option skips duplicate IDs within the input file and conflicts reported by the memory service. It does not guarantee that repeating an import will avoid duplicate memories. For the file format and examples, see Manually creating memories for GitHub Copilot.

Using `copilot workflow run`

Run a dynamic workflow provided by an extension, without opening an interactive session:

Shell

```
copilot workflow run WORKFLOW-NAME [OPTIONS]
```

`WORKFLOW-NAME` is the name registered by the extension, not a path to an extension file. The command starts a run and waits for it to finish, pause, or otherwise stop.

Option	Description
`--args JSON` or `--args @PATH`	Supply the workflow's inputs. Replace "JSON" here with inline JSON, or replace "PATH" with the path to a UTF-8 JSON file. Defaults to `{}` when omitted. The accepted inputs depend on the workflow.
`--result-file PATH`	Write the returned value as JSON to a file instead of including it in standard output. Written only when the workflow completes and returns a result. An existing file is replaced only after the new result has been written successfully.
`-s`, `--silent`	Suppress progress output. The final result is still emitted unless directed to a file. Error and status diagnostics may still appear on standard error.

`--output-format FORMAT`	Set the standard output format to `text` (default) or `json` (JSONL, one JSON record per line). Combine `json` with `--silent` to receive only the final workflow record. Does not change the JSON format of `--result-file`.

Relative argument-file and result-file paths are resolved from the working directory. Without `--silent`, output includes phases and progress messages reported by the workflow.

The command uses the CLI's authentication, model, permission, and sandbox settings. Permission options such as `--allow-tool` and `--allow-url` apply to the workflow's agents. It does not display permission approval prompts.

Prompt and session-mode options cannot be combined with this command. Examples include `-p`, `-i`, `--agent`, `--fleet`, `--autopilot`, `--resume`, and `--continue`. Piped standard input is not used as a prompt or as workflow arguments.

For more information, see Using dynamic workflows and GitHub Copilot CLI programmatic reference.

Using `copilot sandbox ca`

Run `copilot sandbox ca` to manage the sandbox's proxy certificate authority from outside an interactive session—for example, when provisioning managed devices without a TTY. This mirrors the `/sandbox ca` slash commands (`create`, `trust`, `rotate`, `remove`), plus a `status` check.

```
copilot sandbox ca status
copilot sandbox ca create
copilot sandbox ca trust
copilot sandbox ca rotate
copilot sandbox ca remove
```

Subcommand	Description
`copilot sandbox ca status`	Report trust state without changing anything.
`copilot sandbox ca create`	Create the certificate authority if needed, without trusting it.
`copilot sandbox ca trust [CA.PEM]`	Add the authority (default: your own) to OS trust.
`copilot sandbox ca rotate`	Replace the authority, preserving its current trust state.
`copilot sandbox ca remove`	Remove the authority from OS trust; the bundle-based path keeps working.

Run these commands as the user who runs Copilot CLI, not elevated, as `SYSTEM`, or as `root`, because the certificate authority is stored in that user's Copilot CLI home directory. Run `copilot sandbox ca trust --help` for unattended setup guidance. `--config-dir` (or

`COPILOT_HOME`) targets a non-default Copilot CLI home for these commands, the same as the rest of the CLI.

The sessions sidebar

The sessions sidebar is a panel docked beside your current conversation that provides a quick way of working with your local GitHub Copilot CLI sessions.

When the sidebar has focus, the following keyboard shortcuts are available.

Shortcut	Purpose
`←`	From the conversation, open the sidebar and move focus into it (Vim: `h`).
`→`	Move focus back to the conversation; press again to close the sidebar (Vim: `l`).
`↑` / `↓`	Move between sessions in the sidebar (also `k` / `j`).
`Enter`	Open the selected session and return focus to the conversation.
`n`	Start a new session.
`x` twice	Close the selected session.
`↑` then `Tab`	From the top of the sessions list, `↑` moves onto the header buttons (← close sidebar, + new session). `Tab` switches between them.
`?`	Show the sidebar help.

The sidebar also responds to the mouse.

Gesture	Purpose
Click a session	Switch to that session and return to the conversation.
Double-click a session	Switch to that session and stay in the sidebar.
Drag the divider	Resize the sidebar.
Hover the sidebar and scroll	Scroll the session list.
Click a header button	← to close the sidebar, + to start a new session.

Hover over the right side of a session	Show the close button (requires `sidebar.showCloseButton`).

Session status indicators

Each session in the sidebar shows an indicator of its current state.

Indicator	Meaning
Green `●` (steady)	The session is idle. It is waiting for your next prompt.
Purple `●` (pulsing)	A turn is in flight. The agent is actively working.
Blue `●` (steady)	The agent finished a turn while you were in another session, and you have not opened it since. Opening the session clears this unread state.
Yellow `●` (steady)	The agent stopped on an error, or otherwise needs your attention.
Gray `○` (steady)	A resumable session. It is saved, but not currently running.
Yellow `!`	The session needs permission to continue.
Yellow `?`	The session is waiting for you to answer a question or make a choice.

The color names above are a guide. The exact shades depend on your terminal's color scheme and the theme you have set with `/theme`. For example, the `github` theme uses its own fixed colors, and the `colorblind` theme uses blue and yellow in place of green and red. On terminals without full color support, the dots use your terminal's own green, blue, and yellow.

The colors named above relate to the default chat mode. Plan mode and autopilot mode tint some dots differently. The session name's text color also changes in those modes. Because color can shift with your terminal, theme, and mode, the steady or pulsing shape and the `!`, `?`, and `○` markers are the most reliable cues.

Sessions sidebar settings

Use the `/settings sidebar` slash command in an interactive CLI session to adjust these settings for the sessions sidebar.

Setting	Description	Default
`sidebar.enabled`	Allow the sessions sidebar to be displayed.	On

`sidebar.showResumableSessions`	The sidebar includes previous sessions that you can resume. When set to `off`, only active sessions are listed. Changes to this setting take effect the next time Copilot CLI is started.	On
`sidebar.showHeaderButtons`	Show the row of buttons at the top of the sidebar for collapsing it (←) and starting a new session (+).	On
`sidebar.showCloseButton`	As an alternative to clicking `X`, hovering the mouse towards the right of a session in the sidebar displays a **close session** button. Click this, then click again to confirm. The session is permanently deleted, whether or not it was still running. You won't be able to resume a session that's been removed this way.	Off
`sidebar.hoverFocus`	The focus changes from the main CLI tab to the sidebar depending on where the mouse pointer is.	Off
`sidebar.coloredHints`	While the sidebar is open and your conversation is focused, show the **use sidebar** hint in the sidebar's accent color to call attention to it. When off, the same color as other hints is used. While the sidebar is closed, the same color as other hints is always used.	On
`sidebar.accentActiveSession`	Change the session selection highlighting in the sidebar. When on, the session currently open in your conversation gets the main highlighting. When off the session selected with the keyboard up and down arrows (or with the mouse) gets the main highlighting.	On

For more information about the sessions sidebar, see Working with multiple GitHub Copilot CLI sessions.

Global shortcuts in the interactive interface

Shortcut	Purpose
`@ FILENAME`	Include file contents in the context.
`# NUMBER`	Include a GitHub issue or pull request in the context.
`! COMMAND`	Execute a command in your local shell, bypassing Copilot. Enter `!` alone on an empty prompt to enter shell mode for running multiple shell commands in sequence. Press `Esc` or `Ctrl` + `C` on an empty prompt to exit shell mode.
`$`	Type a lone `$` at the prompt and press `Enter` to hand the terminal over to a real interactive shell (`$SHELL` on Unix, `%COMSPEC%` on Windows) rooted at the session's working directory. Unlike `!` shell mode, this suspends the CLI UI entirely, so job control, full-screen apps, tab completion, and colors all work

	natively. Exit the shell (`exit` , or `Ctrl` + `D` on Unix) to return to the CLI. Only activates for a local, trusted, idle session on a real TTY. Can be disabled in enterprise managed settings. Enabled by default. Disable it with the `shellShortcut` setting–see GitHub Copilot CLI configuration directory.
`?`	Open quick help (on an empty prompt). Press again to dismiss and insert a literal `?` .
`Esc`	Cancel the current operation. Press twice to interrupt the running turn, or to stop background agents when the main agent is idle. In a local session, if the model hasn't started answering the turn yet, the second press displays your prompt in the prompt box again for editing.
`Ctrl` + `C`	Cancel operation / clear input. Press twice to exit.
`Ctrl` + `D`	Shutdown.
`Ctrl` + `G`	Edit the prompt in an external editor (`$EDITOR`).
`Ctrl` + `L`	Clear the screen.
`Ctrl` + `Enter` or `Ctrl` + `Q`	Queue a message to send while the agent is busy.
`Ctrl` + `R`	Reverse search through command history.
`Ctrl` + `Space`	Toggle voice dictation on or off (alias for `Ctrl` + `X` then `v`). This only works if your terminal and OS pass it through. You may need to free this in your OS or terminal key bindings. Use `Ctrl` + `X` then `v` as an alternative.
`Ctrl` + `V`	Paste from clipboard as an attachment.
`Alt` + `V`	Paste image from clipboard as an attachment.
`Ctrl` + `X` then `/`	After you have started typing a prompt, this allows you to run a slash command–for example, if you want to change the model without having to retype your prompt.
`Ctrl` + `X` then `e`	Edit the prompt in an external editor (`$EDITOR`).
`Ctrl` + `X` then `b`	Promote the running task or shell command to the background.
`Ctrl` + `X` then `g`	Collapse or expand the autopilot goal panel.
`Ctrl` + `X` then `h`	Hide the current-session sidebar.

`Ctrl` + `X` then `o`	Open the most recent link from the timeline.
`Ctrl` + `X` then `v`	Toggle voice dictation on or off.
`Ctrl` + `X` then `x`	Close the current session.
`Ctrl` + `Z`	Suspend the process to the background (Unix).
`Shift` + `Enter` or `Option` + `Enter` (Mac) / `Alt` + `Enter` (Windows/Linux)	Insert a newline in the input.
`Shift` + `Tab`	Cycle between standard, plan, and autopilot mode.

In local sessions, you can queue prompts, shell commands, and supported slash commands to run in order after the current task finishes. Press `Ctrl` + `Q` to queue the current input while the agent is running. Queued entries display a "pending" label and can be canceled individually before they run.

With an empty prompt box, press `↑` to recall the most recently queued or steering prompt back into the prompt box for editing before it's resubmitted. A "recall" hint appears next to the queue when this is available. Use `Ctrl` + `P` instead for nondestructive navigation through submitted command history.

In a local session, pressing `Esc` twice on a submitted prompt whose turn the model hasn't started answering puts your prompt back in the prompt box and removes it from the conversation. If the model has already started answering, the same second `Esc` press instead interrupts the running turn. Afterward, with an empty prompt box, pressing `↑` restores the prompt to the prompt box.

Timeline shortcuts in the interactive interface

Shortcut	Purpose
`Ctrl` + `F`	Open timeline search.
`Ctrl` + `O`	While there is nothing in the prompt input, this expands recent items in Copilot's response timeline to show more details.
`Ctrl` + `E`	While there is nothing in the prompt input, this expands all items in Copilot's response timeline.
`Ctrl` + `T`	Expand/collapse display of reasoning in responses.
`Page Up` / `Page Down`	Scroll the timeline up or down by one page.

Tasks dialog shortcuts

When the tasks dialog is open (opened via `/tasks`):

Shortcut	Purpose
`↑` / `K`	Move selection up.
`↓` / `J`	Move selection down.
`Enter`	View task details, or teleport into the selected subagent's own session view.
`a`	Toggle showing all nested subagent levels vs. this level's tasks only.
`f`	Toggle showing finished subagents and shells, not just active ones.
`X`	Kill the selected active task.
`R`	Remove the selected finished task.
`B`	Promote the selected sync task to background.
`Esc`	Close the dialog (or go back to the list).

Subagents that spawn their own nested subagents appear as an indented tree; the row you're teleported into is highlighted as "current." While drilled into a subagent's view, you can send it a steering message from the prompt box the same way you would the main session.

Restoring an interrupted session

If a session was still open when its CLI process went away–for example, due to a crash or a machine restart–the next `copilot` startup can offer to restore it, letting you choose which sessions to bring back or start fresh instead. A session whose agent was mid-turn automatically resumes that work once restored.

This restore behavior is temporarily opt-in while its startup dialog is reworked. Set the `COPILOT_ENABLE_INTERRUPTED_SESSION_RESTORE` environment variable to `1` to enable it. See Environment variables.

Session picker shortcuts

When the session picker is open (opened via `/resume` or `--continue`):

Shortcut	Purpose
`↑` / `↓`	Move selection up or down.
`Enter`	Open the selected session.
`s`	Cycle the sort order: relevance → created → name → last used.
`Tab`	Switch between the local and remote tabs.
`x`	Delete the selected session.
`Esc`	Close the picker.

Sessions sort by the following modes:

Mode	Description
`relevance`	Scores sessions by match to the current working directory (default).
`last used`	Most recently modified sessions first.
`created`	Most recently created sessions first.
`name`	Alphabetical by session name; unnamed sessions sort to the end.

Sessions already open in another window float to the top in all non-relevance sort modes. When no working-directory context is available, the `relevance` mode is skipped.

Pre-conversation environment picker

Before you send the first message in a new session, press `Ctrl`+`E` in an empty prompt box to choose where the conversation will run: on your local machine, or in a cloud environment. The environment is provisioned when you send your first message.

> **Note**
>
> The environment picker requires cloud sessions to be available on your account–the same requirement as the `--remote`, `--no-remote`, `--remote-export`, `--no-remote-export`, and `--connect` options. See Command-line options.

Once a conversation has started, `Ctrl`+`E` reverts to its other behaviors: moving the cursor to the end of the line while typing, or expanding all items in the timeline on an empty prompt box. See Navigation shortcuts in the interactive interface and Timeline shortcuts in the interactive interface.

Sidebar and sessions tab shortcuts

The current-session sidebar lets you browse and switch between sessions without leaving the one you're in. With an empty prompt box, `←`/`→` walk a three-state focus cycle: closed, timeline-focused, and sidebar-focused. Disable the sidebar entirely with the `sidebar` setting—see GitHub Copilot CLI configuration directory.

Shortcut	Purpose
`←` (Current tab, closed)	Open the sidebar; the timeline keeps focus.
`←` (timeline-focused)	Move focus into the sidebar.
`→` (sidebar-focused)	Return focus to the timeline.
`→` (timeline-focused)	Close the sidebar.
`Tab` / `Shift`+`Tab`	Switch tabs (**Current**, **Sessions**, and so on).
`↑` / `↓` (sidebar-focused)	Move the selection cursor; `j` / `k` are aliases.
`Enter` (sidebar-focused)	Switch the foreground session to the selected card.
`n` (sidebar-focused)	Spawn a new background session and bring it to the foreground.
`s` (sidebar-focused)	Cycle the sort order: recent → created → name → none.
`x`, `x` (sidebar-focused)	Close the selected card (armed for one keystroke, shown in red, before closing).

The sidebar's divider can be dragged to resize it, and it auto-collapses below a minimum usable width. Each card shows a status dot, the session name, and its Git branch (or working directory when there is no branch). The foreground session's title is highlighted in accent color, and its card carries a persistent subtle fill.

Dots indicate a session's status. For what each dot means, see Session status indicators.

The active sort order (`recent`, `created`, `name`, or `none` for insertion order) persists across restarts.

Mouse actions mirror the keyboard: click a card to switch the foreground session, double-click a card or empty rail space to focus the sidebar, and click the timeline pane while the sidebar is focused to return focus to it.

The **Sessions** tab lists the current session plus your full resumable session history under a "Resumable" divider:

Shortcut	Purpose

`↑` / `↓`	Navigate rows.
`Shift` + `↑` / `↓`	Page up or down.
`Enter`	Resume the selected session.
`n`	Start a new session.
`a`	Cycle the filter scope: all → local → remote (cloud).
`/`	Search live across name, branch or working directory, repository, and session ID.
`x` , `x`	Close or delete the selected row (armed for one keystroke, shown in red, before acting); `Ctrl` + `X` then `x` still works as a two-key alias.
`←` / `→`	Switch tabs.

Remote (cloud) rows in the **Sessions** tab also show online or offline status and the repository.

Pressing `x` twice on a row closes a running session or, for a local resumable row, permanently deletes that session's stored history. Pressing `Esc`, pressing any other key, moving the highlight, or a timeout cancels the pending confirmation.

Diff mode shortcuts

When diff mode is open (entered via `/diff`):

Shortcut	Purpose
`↑` / `k`	Move selection up one line.
`↓` / `j`	Move selection down one line.
`←` / `h`	Jump to the previous file.
`→` / `l`	Jump to the next file.
`Home` / `g`	Jump to the first line.
`End` / `G`	Jump to the last line.
`Page Up`	Scroll up one page.
`Page Down`	Scroll down one page.
`Ctrl` + `U`	Scroll up half a page.

`Ctrl` + `D`	Scroll down half a page.
`Click`	Select the clicked diff line (requires mouse support).
Mouse scroll	Scroll up or down.
`Alt` / `Option` +scroll	Scroll one line at a time for fine-grained control.
`c`	Add or edit a comment on the selected line.
`s`	Show comments summary (when comments exist).
`b`	Toggle between unstaged changes and branch diff.
`w`	Toggle hiding whitespace-only changes.
`Enter`	Submit all comments (when comments exist).
`r`	Refresh the diff (remote sessions only).
`Esc` / `Ctrl` + `C`	Exit diff mode.

Navigation shortcuts in the interactive interface

Shortcut	Purpose
`Ctrl` + `A`	Move to beginning of the line (when typing).
`Ctrl` + `B`	Move to the previous character.
`Ctrl` + `E`	Move to end of the line (when typing).
`Ctrl` + `F`	Move to the next character.
`Ctrl` + `H`	Delete the previous character.
`Ctrl` + `K`	Delete from cursor to end of the line. If the cursor is at the end of the line, delete the line break.
`Ctrl` + `U`	Delete from cursor to beginning of the line.
`Ctrl` + `W`	Delete the previous word.
`Home`	Move to the start of the current visual line.
`End`	Move to the end of the current visual line.
`Ctrl` + `Home`	Move to the start of the text.

`Ctrl` + `End`	Move to the end of the text.
`Alt` + `←` / `→` (Windows/Linux) `Option` + `←` / `→` (Mac)	Move the cursor by a word.
`↑` / `↓`	Navigate the command history. With an empty prompt box, `↑` recalls the most recently queued or steering prompt instead, if one is available.
`Ctrl` + `P`	Navigate submitted command history nondestructively, without triggering the queued-prompt recall behavior.
`Tab` / `Ctrl` + `Y`	Accept the current inline completion suggestion.

Slash commands in the interactive interface

These are the slash commands you can use from within an interactive CLI session. A subset of these slash commands is available to clients that use the CLI via its ACP server. For more information, see Copilot CLI ACP server.

Command	Purpose
`/add-dir PATH`	Allow file access to a directory and load its skills (`.github/` , `.agents/` , `.claude/`) and agents (`.github/` , `.claude/`) as trusted configurations. When sandboxing is enabled, also grants the sandbox access to the directory for the current session.
`/after [DELAY PROMPT]` , `/after`	Schedule a non-recurring prompt, skill, or schedulable slash command for the current session (for example, `/after 30m remind me the time` or `/after 1h /chronicle standup`). With no arguments the schedule manager is displayed. Only available in experimental mode.
`/agent`	Browse and select from available agents (if any). See Custom agents.
`/app`	Open the current session in the GitHub Copilot app (requires version 1.1.3 or later), or show the download URL if the app is not installed.
`/ask [QUESTION]` , `/btw [QUESTION]`	Ask a quick side question without adding to the conversation history. The response renders as Markdown. In the answer dialog, press `Ctrl` + `E` to expand it and `Ctrl` + `Y` to copy the full answer. Enter the command without a question to reopen the

	last side question answer in the current session. See Asking a side question in GitHub Copilot CLI.
`/allow-all [on|off|show]`, `/yolo [on|off|show]`	Enable all permissions (tools, paths, and URLs). This is an alias for `/permissions allow-all`; see the `/permissions` row for the canonical command and its subcommands.
`/autopilot [OBJECTIVE]`, `/goal [OBJECTIVE]`	Start or refocus autopilot mode, optionally with an explicit objective (for example, `/goal Refactor the auth module`). Without an objective, autopilot infers intent from context, and the status panel shows your last prompt as the inferred objective. You can cap AI-credit spend for the objective by using `--max-ai-credits N` (for example, `/goal Refactor the auth module --max-ai-credits 5`). When the cap is reached, autopilot pauses and opens a panel reporting credits used against the cap. Enter a new amount to resume with a fresh credit window, or dismiss the panel to stay paused. You can also resume a paused objective yourself, without the panel, by running the option on its own with no objective text–for example, `/goal --max-ai-credits 5`. This is the same action the panel performs: it opens a fresh window of the credits you specify (the full new cap, not an increment) and continues the objective. `/goal on` and `/goal off` toggle autopilot mode without setting an objective and don't accept `--max-ai-credits`. An active goal renders as a pinned panel above the prompt box, showing the objective, credits used, and todo progress. The panel auto-collapses to a single identity row on short terminals (below 30 rows) and expands above that threshold; press `Ctrl`+`X` then `g` to override the automatic sizing by hand.
`/changelog [summarize] [VERSION|last N|since VERSION]`, `/release-notes [summarize] [VERSION|last N|since VERSION]`	Display the CLI changelog. Optionally specify a version, a count of recent releases, or a starting version. Add the keyword `summarize` for an AI-generated summary.
`/chronicle <standup|tips|improve|reindex|skills create|skills review|skills status>`	Session history tools and insights. The `skills` subcommands draft, review, and track the status of repository skill proposals generated from observed usage. See Using GitHub Copilot CLI session data.
`/clear [PROMPT]`, `/new [PROMPT]`, `/reset [PROMPT]`	Start a new conversation. `/new worktree` starts an empty session in a new Git worktree instead of clearing the current one, leaving the current conversation and its working directory unchanged.
`/clikit [COMPONENT]`	Preview CLI business components (for example, quota info).

`/collect-debug-logs [file|gist|share] [PATH]`	Collect debug logs to an archive, a GitHub gist, or upload them to GitHub (`share`). See Collecting and sharing debug logs.
`/compact [FOCUS-INSTRUCTIONS]`	Summarize the conversation history to reduce context window usage. Optionally provide focus instructions to steer the summary–for example, `/compact focus on the auth module` . See Managing context for GitHub Copilot.
`/context`	Show the context window token usage and visualization. See Managing context for GitHub Copilot.
`/copy`	Copy the last response to the clipboard.
`/cwd` , `/cd [PATH]`	Change the working directory or display the current directory.
`/delegate [PROMPT]`	Delegate changes to a remote repository with an AI-generated pull request. See Delegating tasks to Copilot.
`/diagnose [PROMPT]` , `/diagnose`	Analyze the current session log for errors, unexpected behavior, and other issues. Optionally include a custom prompt to focus the diagnosis on a specific problem.
`/diff`	Review changes in the current directory; auto-switches to branch diff when the working tree is clean (experimental).
`/downgrade VERSION`	Download and restart into a specific CLI version. Available for team accounts.
`/env`	Show loaded environment details (instructions, MCP servers, skills, agents, hooks, plugins, LSPs, extensions).
`/every [INTERVAL] PROMPT` , `/every`	Schedule a recurring prompt, skill, or schedulable slash command for the current session (for example, `/every 1h run tests` or `/every 1d /chronicle standup`). With no arguments the schedule manager is displayed. Only available in experimental mode.
`/exit` , `/quit`	Close the current session. If other sessions are running, this foregrounds the newest remaining one instead of quitting. Quits the CLI only when it is the last open session. `/exit print` always tears down the CLI and offers to dump the transcript. `/exit` and `/quit` (including their `print` variant) can also be queued with `Ctrl` + `Q` while the agent is running

	–the CLI closes the session once the current turn, and any queued shell command ahead of it, finishes.
`/extensions [manage|mode]`, `/extension`	Manage CLI extensions. Only available in experimental mode.
`/experimental [on|off|show]`	Toggle, set, or show experimental features.
`/feedback`, `/bug`	Provide feedback about the CLI. Can offer a feedback-with-logs route; see Collecting and sharing debug logs.
`/fleet [PROMPT]`	Enable parallel subagent execution of parts of a task. See Speeding up task completion with the /fleet command.
`/help`	Show the help for interactive commands.
`/ide`	Connect to an IDE workspace. See Connecting GitHub Copilot CLI to VS Code.
`/init [suppress|INSTRUCTIONS]`	Initialize Copilot custom instructions and agentic features for this repository. Text after the command guides what is written. `suppress` hides the startup reminder for this repository. See Project initialization for Copilot.
`/instructions`	View and toggle custom instruction files.
`/keep-alive [on|off|busy|DURATION]`, `/caffeinate [on|off|busy|DURATION]`	Prevent the machine from going to sleep: while a CLI session is active, while the agent is busy, or for a defined length of time. Accepts durations like `30`, `30m`, `2h`, `1d` (bare numbers default to minutes).
`/limits`	Open the interactive response limits dialog.
`/limits set max-ai-credits VALUE`	Set a soft maximum for AI Credits allowed per response. Response limits are soft limits that reset for each user message. See Setting an AI credit session limit in GitHub Copilot CLI.
`/limits unset [max-ai-credits|all]`	Remove a specific response limit, or all response limits.
`/list-dirs`	Display all of the directories for which file access has been allowed.
`/login`	Log in to Copilot.
`/logout`	Log out of Copilot.

`/lsp [show|test|reload|logs|help] [SERVER-NAME]`	Manage the language server configuration. The `logs` subcommand opens the live LSP services log panel.
`/mcp [config|list|show|add|edit|delete|disable|enable|auth|reload|search] [SERVER-NAME]`	Manage the MCP server configuration. With no subcommand, or with `config`, the plugins dashboard opens pinned to the MCP server list; the add, edit, and authenticate forms open inside that dashboard too, so closing a form returns you to the server list. Use `show` or `show SERVER-NAME` to display all configured servers or open one server's details directly, including its available tools, and to enable or disable it. For a plugin-provided server, `show SERVER-NAME` also displays the source attribution (for example, `Source: Plugin my-plugin (1.2.0)`). `list` (alias `ls`) prints a plain-text list of configured servers with connection status and live state. Bare `/mcp`, `config`, `show`, and `list` (alias `ls`) are read-only or open the dashboard, so they can run while the agent is busy processing a turn. The mutating subcommands (`add`, `edit`, `delete`, `disable`, `enable`, `auth`, `reload`, and `search`) are blocked until the turn finishes. `edit <name>` rejects a workspace-sourced server (one defined in a repository's `.mcp.json`) instead of opening the user-tier wizard, since saving would silently create a same-name user entry that the workspace one still shadows. The error names the file to edit directly. `delete <name>` reports the same file when asked to remove a workspace-sourced server. Sandboxed local servers show a `connected (sandboxed)` status. See Adding MCP servers for GitHub Copilot CLI.
`/model [--session|--global|--repo|--local] [MODEL|auto TIER]`, `/models`	Select the AI model you want to use, or choose **Auto**. Model IDs are case-insensitive, and the canonical ID is saved. By default (or with `--session`, alias `-s`), changes the model, reasoning effort, or context window for the current session only, without touching saved settings. `--repo` / `--local` pins the default model in repository settings instead; `--global` (or `/config model`) sets the default for future sessions. Press `Tab` on a model with a long-context variant to toggle its Context column between the default and long-context window. The picker groups models into sections—press `Shift`+`Tab` to cycle grouping between recommended (Recent, Recommended, New, and other models), vendor, and category. The picker marks your effective managed default model with the default badge when it is selectable. A model with vendor-specific data retention terms shows a data retention warning banner with a link to the vendor's policy.

	Usable mid-turn: a change requested while the agent is running is queued as a cancellable (`Ctrl`+`C`) command and applied once the current turn finishes, instead of switching the live model mid-request. Use `/model auto TIER` (`efficiency`, `balance`, or `intelligence`) to select a specific Auto routing tier directly, including from the "switch" action on an Auto tier recommendation hint. When Auto picks a model, the timeline shows the reason for that choice whenever the API provides one. See About GitHub Copilot auto model selection.
`/permissions [default|assisted|allow-all|show]`	Switch between permission modes (`default`, `assisted`, `allow-all`), or show the current mode (`show`). This is the canonical command for permission mode changes; `/allow-all` and `/yolo` remain supported as aliases. For information about assisted permissions, see Allowing safe commands to run without manual approval.
`/permissions reset`	Reset all in-memory tool and path approvals for the current session (re-prompt on next use).
`/plan [PROMPT]`	Create an implementation plan before coding.
`/plugin`	Open the plugins dashboard showing your installed plugins. Select a plugin and press `Enter` for its details and for actions such as enabling, disabling, updating, or uninstalling it. Use the `←` and `→` keys to switch between the **Installed**, **Online**, and **Marketplace** views. The dashboard shows only plugins—use `/mcp` for MCP servers and `/skills` for skills. See Plugins for GitHub Copilot.
`/plugin install SOURCE`	Install a plugin from a marketplace spec, GitHub repository, git URL, or local path.
`/plugin update PLUGIN[@MARKETPLACE]`	Update an installed plugin.
`/plugin uninstall PLUGIN[@MARKETPLACE]` (aliases `remove`, `rm`)	Uninstall a plugin.
`/plugin list` (alias `ls`)	List installed plugins. Team plugin assignments set optional defaults for team members. Plugins assigned as default-enabled appear as enabled, and their skills load, unless you disable them or managed settings override the default. In contrast, plugins enforced through managed `enabledPlugins`, including enterprise team settings, cannot be disabled locally. See GitHub Copilot CLI plugin reference.
`/plugin marketplace add SOURCE`	Add a marketplace.

`/plugin marketplace remove NAME`	Remove a marketplace.
`/plugin marketplace list`	List registered marketplaces.
`/plugin marketplace browse NAME`	Browse plugins in a marketplace.
`/plugin marketplace update [NAME]` (alias `refresh`)	Re-fetch a marketplace's plugin catalog, or every registered marketplace when no name is given.
`/pr [view|create|fix|auto|automerge]`	Manage pull requests for the current branch. `auto` drives the pull request to green and stops; `automerge` (alias: `agentmerge`) drives the pull request to green and merges it. When the repository has a pull request template (`.github/pull_request_template.md` or a file under `.github/PULL_REQUEST_TEMPLATE/`), `create` follows it by default—preserving headings, comments, and checklist items, and filling in accurate details instead of writing a generic body. Ask Copilot to skip the template if you don't want it applied. See Managing pull requests with the /pr command.
`/refine TEXT`	Rewrite a roughly composed prompt into a clear one for review. Run with no arguments (via `Ctrl`+`X` then `/refine`) to clean up the current input box. Can be particularly useful for prompts entered by speaking.
`/remote [on|off]`	Show the remote control status (if no argument provided), enable remote steering (`on`), or end the remote connection (`off`). See Steering a GitHub Copilot CLI session from another device.
`/rename [NAME]`	Rename the current session (auto-generates a name if omitted; alias for `/session rename`).
`/research TOPIC`	Run a deep research investigation using GitHub search and web sources. See Researching with GitHub Copilot CLI.
`/reset-allowed-tools`	Reset permissions for the current session. This clears session tool approvals, returns the permission mode to manual (turning off allow-all or assisted permissions if either is on), and clears the approvals saved for the current location. Explicit startup rules, such as `--allow-tool`, `--deny-tool`, `--allow-url`, and `--deny-url`, remain part of the session's baseline.
`/restart`	Restart the CLI, restoring all of the current process's live sessions, not only the foreground one. If the target CLI version can't restore multiple sessions, you're prompted to continue with only the

	foreground session or cancel. Sessions started in a Git worktree with `-w` or `--worktree` resume in that same worktree. The model selected with `/model` is preserved.
`/resume [SESSION-ID]`, `/continue [SESSION-ID]`	Switch to a different session by choosing from a list (optionally specify a session ID).
`/review [PROMPT]`	Run the code review agent to analyze changes. See Requesting a code review with GitHub Copilot CLI.
`/rubber-duck [PROMPT]`	Consult the rubber duck agent for a second opinion on plans, code, and tests. See Getting a second opinion with the rubber duck agent.
`/sandbox [config|status|policy [COMMAND]|enable|disable]`	Manage sandbox restrictions on filesystem and network access. `config` (or bare `/sandbox`) opens the settings dialog, which suggests possible environment secrets and lets you add masking hosts before saving. The dialog's **General** tab has separate options for enabling sandboxing for the current session and to set the default for new sessions. Changing either option leaves the other unchanged. `/sandbox enable session` and `/sandbox disable session` change sandboxing for the current session without changing your saved default. `status` shows whether sandboxing is enabled for the current session. When sandboxing is enabled, `policy` shows path grants and restrictions grouped by source and access type, network settings, developer-tool access, and sandbox capabilities. Optionally add a command, such as `/sandbox policy npm install`, to inspect its developer-tool access without running it. `enable` and `disable` normally update your saved setting. If your organization requires sandboxing, `disable` is refused unless bypass is allowed. In that case, disabling applies only to the current session.
`/sandbox ca [create|trust|rotate|remove]`	Manage the sandbox's proxy certificate authority. `create` writes it without trusting it. `trust` adds it to OS trust. `rotate` replaces it while preserving its trust state. `remove` drops OS trust. See Using `copilot sandbox ca` to run the same operations outside a session, for example on managed devices.
`/search [QUERY]`, `/find [QUERY]`	Search the conversation timeline.
`/security-review [PROMPT]`	Run a focused security review of active local code changes and return prioritized vulnerability findings with remediation suggestions. This command is not a full repository security audit.

`/session [info|checkpoints [n]|files|plan|rename [NAME]|cleanup|prune|delete [ID]|delete-all]`, `/sessions [info|checkpoints [n]|files|plan|rename [NAME]|cleanup|prune|delete [ID]|delete-all]`	Show session information and manage sessions. The `info` subcommand shows session details including the session link (when available). Subcommands: `info`, `checkpoints`, `files`, `plan`, `rename`, `cleanup`, `prune`, `delete`, `delete-all`.
`/settings [--repo|--local] [show KEY|KEY|KEY VALUE]`, `/config [--repo|--local] [show KEY|KEY|KEY VALUE]`	Open the settings dialog, open it focused on a specific setting (`KEY`), set a setting inline (`KEY VALUE`), or display a setting's current value (`show KEY`). `show` masks secret-named values (for example, tokens or API keys nested under a setting) instead of printing them in clear text. The dialog shows **User**, **Repo**, **Repo (local)**, and **Problems** tabs–switch with `Tab` / `Shift` + `Tab` ; a setting overridden in another scope shows a badge naming which scope wins. The **Problems** tab is a cross-scope view of settings that need attention (for example, unknown or invalid keys); its label shows a count, such as `Problems (2)` , when any scope has an issue, and otherwise just reads `Problems` . The dialog also includes a global search that finds visible settings, areas, and local resources. Add `--repo` or `--local` to target `.github/copilot/settings.json` or `.github/copilot/settings.local.json` instead of the user settings file–for example, `/settings --repo model gpt-6-astra` . Only repo-overridable keys can be set this way. Rows governed by an active organization or MDM managed policy render read-only with a `(managed)` tag. See Changing settings with the /settings command.
`/share [link|off|file|html|gist|research] [...]`, `/export [...]`	Share the current session. With no subcommand, opens a sharing menu without sharing or exporting anything. `link` shares the session and prints a shareable GitHub link. `off` or `link off` stops link sharing. `file [session|research] [PATH]` exports to a Markdown file. `html [session|research] [PATH]` exports to an HTML file. `gist [session|research]` creates a GitHub gist. `research [PATH]` exports the research report.
`/skills`	Open the plugins dashboard on the Skills tab.
`/skills list`	List all available skills.
`/skills info NAME`	Show details of a specific skill.
`/skills add [--project] <FILE|URL|DIRECTORY>`	Add a skill from a file, URL, or directory; `--project` scopes a file or URL install to this repository instead of your user account.

`/skills remove <NAME|DIRECTORY>`	Remove a skill by name, or unregister a custom skill directory.
`/skills reload`	Reload skills from all directories. See Adding agent skills for GitHub Copilot CLI.
`/statusline`, `/footer`	Configure which items appear in the status line.
`/subagents`, `/agents`	Configure default and per-agent subagent models. For an agent with `modelPolicy: "required"` — whether set in the agent's own definition or in the `subagents` settings override—the picker shows a locked **Model** entry and a **Model enforcement** row reading "required - cannot be overridden" (or "required by agent definition" when the agent definition itself sets the policy). Switch the policy back to `"preferred"` from the picker to allow overrides again, unless the agent definition requires it. See GitHub Copilot CLI configuration directory.
`/tasks`	View and manage tasks (subagents and shell commands).
`/terminal-setup`	Configure the terminal for multiline input support (`Shift`+`Enter` and `Ctrl`+`Enter`).
`/theme [default|github|dim|high-contrast|colorblind]`	View or set the color mode.
`/tuikit [colors|icons|select|tabbar]`	Preview TUIkit design-system components and color tokens.
`/undo`, `/rewind`	Open the rewind picker to roll the session back to an earlier user turn. Choose between: **Conversation only** (roll back the conversation, leaving files as they are) or **Conversation + files** (also restore the files Copilot changed in that turn, and later discarded turns, to their pre-change contents, skipping any you have since edited yourself). File changes are tracked per turn across editing tools, shell commands, and sub-agents, so Git is not required. A rewind cannot be undone: later turns are removed from the session. Use `/fork` first to keep a copy.
`/update`, `/upgrade`	Update the CLI to the latest version. `/update stable` or `/update prerelease` also saves the selected channel for future updates.
`/usage`	Display session usage metrics and statistics, including per-model token totals. For token-based-billing accounts, each model's row also shows its own AI credit consumption (for example, `1 AIC`).

	Billed models with no token counts show credits only.
`/user [show|list|switch]`	Manage signed-in accounts.
`/version`	Display version information and check for updates.
`/vim`	Toggle Vim mode for the prompt box, enabling Vim-style modal editing: motions (for example, `hjkl`, `w`, `b`, `e`, `0`, `$`, `gg`, `G`), character search (`f` / `F` / `t` / `T` / `;` / `,`), insert commands (`i` / `a` / `o`), edit commands (`r` / `~` / `J` / `x` / `D` / `C`), operators (`d` / `c` / `y`), yank and put (`y` / `p` / `P`), repeat (`.`), undo and redo (`u` / `Ctrl`+`R`), counts, and `Esc` to return to normal mode. Also configurable with the `editorMode` setting. See GitHub Copilot CLI configuration directory.
`/voice [on|off|models|devices]`	Toggle voice mode, browse available voice models, or choose the input device (microphone).
`/workflows`	Observe workflow runs, phases, agents, and progress. Requires a token-based-billing plan. See Using `copilot workflow run` to run a registered workflow directly.
`/fork [NAME]`, `/branch [NAME]`	Fork the current session into a new session, optionally with a name. Usable while the agent is running–the source session keeps working in the background. `/fork worktree` forks the current session, preserving its conversation context, into a new Git worktree branched off `HEAD`.
`/worktree [branch|task]`	Create a new Git worktree and switch to it, leaving uncommitted changes behind in the current worktree. Pass a branch name, a task description (multiline supported, used as the opening prompt in the new worktree), or omit the argument to auto-generate a branch name from the conversation. By default, branches off the current checkout (`HEAD`); set the `worktreeBaseRef` setting to `"defaultBranch"` to branch off the remote default branch instead. See GitHub Copilot CLI configuration directory. Requires a Git repository.
`/worktree new [PROMPT]`	Deprecated–use `/new worktree` instead. Starts a new conversation in a new Git worktree, leaving the current conversation and its working directory unchanged. Optionally provide the first prompt. `new` is reserved as the subcommand keyword and can't be used as a literal branch name. Follows the same `worktreeBaseRef` setting as `/worktree`.

`/move [branch|task]`	Move uncommitted changes into a new Git worktree and switch to it. Pass a branch name, a task description (multiline supported, used as the opening prompt in the new worktree), or omit the argument to auto-generate a branch name from the conversation. Requires a Git repository.

For a complete list of available slash commands enter `/help` in the CLI's interactive interface.

In the schedule manager opened by a bare `/every` or `/after`, use `↑`/`↓` to select an entry and `x` to remove it. Schedules can only be added from the prompt input using `/every` or `/after` with arguments—the dialog itself is read-and-remove only.

The footer shows an "N scheduled" indicator by default whenever the session has active schedules, even if `/every` and `/after` are not enabled for you.

`/plugin` flags an installed plugin or marketplace when a newer version is available upstream, and offers an **Update** action from the dashboard.

`/mcp list` / `ls` and `/plugin list` / `ls` (including bare `/plugin`) are read-only and can run while the agent is busy processing a turn. All other `/mcp` and `/plugin` subcommands are blocked until the turn finishes.

> **Note**
>
> The experimental `/plugins` command was removed. Its resources have been moved to `/plugin`, `/mcp`, and `/skills`. Use `/subagents` and `/instructions` for agents and instructions.

Collecting and sharing debug logs

Use `/collect-debug-logs [file|gist|share] [PATH]` to gather debug logs for troubleshooting:

Target	Description
`file`	Save the logs to a local archive at `PATH` (default location used when `PATH` is omitted).
`gist`	Upload the logs to a secret GitHub gist.
`share`	Save a local archive, then upload it to GitHub.

`/collect-debug-logs share`, and the feedback-with-logs route offered from `/feedback`, first save a local archive, then ask for fresh consent (defaulting to **No**) before uploading it to GitHub. Uploading is currently available only to GitHub staff and internal users, and isn't generally available yet. Users outside GitHub continue to use the existing archive and gist routes, and the existing public feedback route.

Command-line options

Option	Purpose
`--add-dir=PATH`	Allow file access to a directory and load its skills (`.github/` , `.agents/` , `.claude/`) and agents (`.github/` , `.claude/`) as trusted configurations (can be used multiple times). A relative path resolves against the session working directory (the `--resume` , `--worktree` , or `-C` directory), regardless of option order. The path must be an existing directory–a missing path or a regular file fails startup immediately with a flag-specific error, the same way in every mode.
`--add-github-mcp-tool=TOOL`	Add a tool to enable for the GitHub MCP server, instead of the default CLI subset (can be used multiple times). Use `*` for all tools.
`--add-github-mcp-toolset=TOOLSET`	Add a toolset to enable for the GitHub MCP server, instead of the default CLI subset (can be used multiple times). Use `all` for all toolsets.
`--additional-mcp-config=JSON`	Add an MCP server for this session only. The server configuration can be supplied as a JSON string or a file path (prefix with `@`). A relative `@file` path resolves against the session working directory (the `--resume` , `--worktree` , or `-C` directory), regardless of option order. Augments the configuration from `~/.copilot/mcp-config.json` . Overrides any installed MCP server configuration with the same name. See Adding MCP servers for GitHub Copilot CLI.
`--agent=AGENT`	Specify a custom agent to use. See Custom agents.
`--allow-all`	Enable all permissions (equivalent to `--allow-all-tools --allow-all-paths --allow-all-urls`).
`--allow-all-mcp-server-instructions`	Include initialization instructions from all MCP servers in the system prompt. By default, only allowlisted server instructions are included up front; other servers' instructions are retrieved on demand.
`--allow-all-paths`	Disable file path verification and allow access to any path.
`--allow-all-tools`	Grant blanket tool permissions without a model safety review or individual approval prompts. Deny rules and enterprise restrictions still apply (env: `COPILOT_ALLOW_ALL`).

`--allow-all-urls`	Allow access to all URLs without confirmation.
`--allow-tool=TOOL ...`	Tools the CLI has permission to use. Will not prompt for permission. For multiple tools, use a quoted, comma-separated list. See Allowing and denying tool use.
`--allow-url=URL ...`	Allow access to specific URLs or domains. For multiple URLs, use a quoted, comma-separated list.
`--acp`	Start as Agent Client Protocol server.
`--assisted-approval`	Turn on assisted permissions for the run, so that a safety review can approve eligible permission requests. Only valid for non-interactive runs–that is, runs that use `-p` or `--prompt`, and runs in an environment with no terminal attached. When successfully enabled, takes precedence over `--allow-all-tools`, but does not affect `--allow-all-paths` or `--allow-all-urls` (env: `COPILOT_ASSISTED_APPROVAL`). See Allowing safe commands to run without manual approval.
`--attachment PATH`	Attach a file to the initial prompt (can be used multiple times). Image files are accepted, but sending them successfully requires the selected model and organization policy to allow vision input.
`--autopilot`	Enable autopilot continuation–the agent keeps working until `task_complete` is called, then returns to interactive mode. See Using Autopilot mode in GitHub Copilot CLI.
`--available-tools=TOOL ...`	Only these tools will be available to the model. For multiple tools, use a quoted, comma-separated list. See Allowing and denying tool use.
`--banner`, `--no-banner`	Show or hide the startup banner.
`--bash-env`	Enable `BASH_ENV` support for bash shells.
`-C DIRECTORY`	Change working directory before doing anything else.
`--connect[=SESSION-ID]`	Connect directly to a remote session (optionally specify a session ID or task ID). Conflicts with `--resume` and `--continue`.
`--context TIER`.	Set the context window tier for tiered-pricing models (overrides the persisted setting and is honored in fresh interactive sessions). Choices: "default", "long_context".

`--config-dir=DIRECTORY`	This option for setting the configuration directory is deprecated. Use the `COPILOT_HOME` environment variable instead.
`--continue`	Resume the most recent session in the current working directory, falling back to the globally most recent session. Conflicts with `--resume`.
`--deny-tool=TOOL ...`	Tools the CLI does not have permission to use. Will not prompt for permission. For multiple tools, use a quoted, comma-separated list.
`--deny-url=URL ...`	Deny access to specific URLs or domains, takes precedence over `--allow-url`. For multiple URLs, use a quoted, comma-separated list.
`--disable-builtin-mcps`	Disable the built-in server set for this run. This does not disable every plugin-provided server. See Built-in MCP servers for the affected servers and how to disable computer use.
`--disable-client-telemetry`	Disable telemetry sent by the CLI. Does not affect server-side telemetry.
`--disable-mcp-server=SERVER-NAME`	Disable a specific MCP server (can be used multiple times).
`--disallow-temp-dir`	Prevent automatic access to the system temporary directory.
`--effort=LEVEL`, `--reasoning-effort=LEVEL`	Set the reasoning effort level (`low`, `medium`, `high`, `xhigh`, `max`). `max` is the highest-depth tier for Anthropic models.
`--enable-all-github-mcp-tools`	Enable all GitHub MCP server tools, instead of the default CLI subset. Overrides the `--add-github-mcp-toolset` and `--add-github-mcp-tool` options.
`--enable-mcp-server=SERVER-NAME`	Re-enable an MCP server that's disabled in your settings, for this session only (can be used multiple times). The change is not persisted to your configuration.
`--enable-memory`	Enable memory in prompt mode (disabled by default).
`--enable-reasoning-summaries`	Deprecated compatibility option (accepted but ignored). Detailed reasoning summaries for supporting OpenAI models are shown by default. Toggle them with `Ctrl`+`T`.

`--excluded-tools=TOOL ...`	These tools will not be available to the model. For multiple tools, use a quoted, comma-separated list.
`--experimental`	Enable experimental features (use `--no-experimental` to disable).
`--extension-sdk-path DIRECTORY`	Override the bundled `@github/copilot-sdk` injected into extension subprocesses with a local `copilot-sdk/` folder. Invalid paths fall back to the bundled SDK.
`--fleet`	Run the prompt in fleet mode (parallel subagent orchestration). Combine with `-i`, `-p`, or piped standard input. This requires a prompt from one of those sources, and has no effect on its own. This is equivalent to prefixing an interactive prompt with the `/fleet` slash command. With `-p` or piped input it dispatches the same fleet orchestration non-interactively. See Speeding up task completion with the /fleet command.
`-h`, `--help`	Display help.
`-i PROMPT`, `--interactive=PROMPT`	Start an interactive session and automatically execute this prompt.
`--log-dir=DIRECTORY`	Set the log file directory (default: `~/.copilot/logs/`). A leading `~` expands to your home directory.
`--log-level=LEVEL`	Set the log level (choices: `none`, `error`, `warning`, `info`, `debug`, `all`, `default`).
`--max-ai-credits=CREDITS`	Set a soft maximum for AI Credits allowed for each response. The limit resets per user message and can be adjusted mid-session with `/limits`. See Setting an AI credit session limit in GitHub Copilot CLI.
`--max-autopilot-continues=COUNT`	Maximum number of automatic continuations outside an active autopilot objective (default: 5). Must be a non-negative integer. Malformed values (`NaN`, negative, fractional, or scientific notation) are rejected. See Using Autopilot mode in GitHub Copilot CLI.
`--mode=MODE`	Set the initial agent mode (choices: `interactive`, `plan`, `autopilot`). Combine with `--plan` (as `--plan --mode autopilot`) for plan-then-autopilot: the session starts in plan mode and automatically progresses to autopilot mode once the plan is ready, instead of waiting for a human to approve it. Any other `--plan --mode MODE` pairing is rejected.

`--model=MODEL`	Set the AI model you want to use. Pass `auto` as the value to let Copilot pick the best available model automatically. An explicit `--model` takes precedence over a custom agent's `model` unless the agent sets `model-policy: required`. See About GitHub Copilot auto model selection.
`--mouse[=VALUE]`	Enable or disable mouse support in the interactive interface. VALUE can be `on` (default) or `off`. When enabled, the CLI captures mouse events—scroll wheel, clicks, and so on—to navigate its own interface, such as scrolling the timeline or clicking tabs. When disabled, the terminal's native mouse behavior, such as text selection and scrollback, is preserved. When you set this option explicitly, the value is persisted to your configuration file.
`-n NAME`, `--name=NAME`	Set a name for the new session. Used by `--resume` and `/resume` to find sessions by name.
`--no-ask-user`	Disable the `ask_user` tool (the agent works autonomously without asking questions).
`--no-auto-update`	Disable automatic CLI updates and prevent a newer cached version from running instead of the installed version.
`--no-bash-env`	Disable `BASH_ENV` support for bash shells.
`--no-color`	Disable all color output.
`--no-custom-instructions`	Disable loading of custom instructions from `AGENTS.md` and related files.
`--no-experimental`	Disable experimental features.
`--no-mouse`	Disable mouse support.
`--no-remote`	Removed. Use `/remote off` during an interactive session to end its remote connection.
`--no-remote-export`	Renamed to `--sync-session=off`.
`--output-format=FORMAT`	FORMAT can be `text` (default) or `json` (outputs JSONL: one JSON object per line).
`-p PROMPT`, `--prompt=PROMPT`	Execute a prompt programmatically (exits after completion, or with a nonzero status if the run is interrupted). The exit summary includes a `copilot --resume=SESSION-ID` hint for continuing the session. Use `@` followed by a file or directory path in the prompt to include it as context, for example

	`copilot -p "Explain @src/app.js"`. See Running GitHub Copilot CLI programmatically.
`--plan`	Start in plan mode. Shorthand for `--mode plan`. Cannot be combined with `--autopilot`. Can be combined with `--mode autopilot` for plan-then-autopilot; any other `--mode` value is rejected.
`--plain-diff`	Disable rich diff rendering (syntax highlighting via the diff tool specified by your Git config).
`--plugin-dir=DIRECTORY`	Load a plugin from a local directory (can be used multiple times). A relative path resolves against the session working directory (the `--resume`, `--worktree`, or `-C` directory), regardless of option order. Agents contributed by a `--plugin-dir` plugin are available in server-mode (`--server`) sessions as well as interactive and `-p` sessions.
`--remote`	Not supported for interactive startup or with `-p`. Use `/remote on` during an interactive session instead. The flag remains supported for legacy SDK server startup with `--server` or `--headless`. See Steering a GitHub Copilot CLI session from another device.
`--remote-export`	Renamed to `--sync-session=on`.
`-r`, `--resume[=VALUE]`	Resume a previous interactive session by choosing from a list. Optionally specify a session ID, ID prefix, or session name. Name matching is exact and case-insensitive; falls back to the auto-generated summary when no explicit name matches. Conflicts with `--continue`. Bare `--resume` (no value) shows an interactive session picker, which requires a TTY. If multiple sessions exist and the picker can't be shown (for example under `-p`, a non-TTY `-i`, or piped stdin), the CLI exits with an error instead of silently starting a new session—pass an explicit `--resume=SESSION-ID` or use `--continue`.
`-s`, `--silent`	Output only the agent response (without usage statistics), useful for scripting with `-p`.
`--screen-reader`	Enable screen reader optimizations for this session only and offer to save the preference to your settings.
`--secret-env-vars=VAR ...`	Redact an environment variable from shell and MCP server environments (can be used multiple times). For multiple variables, use a quoted, comma-separated list. The values in the `GITHUB_TOKEN` and

	`COPILOT_GITHUB_TOKEN` environment variables are redacted from output by default.
`--session-id ID`	Use an exact session or task ID when you do not want `--resume`'s broader matching by ID prefix or session name. If the ID matches an existing session or task, that session or task is resumed. If nothing matches, a new session is created only when the value is a valid UUID. Names and ID prefixes do not create new sessions. Do not combine this option with other session-selection or session-starting options such as `--resume`, `--continue`, or `--connect`, because they compete to decide which session to open or create.
`--trust-folder`	Trust the working directory's hooks, workspace MCP servers, and repository plugin settings for one new non-interactive session (`-p` or piped standard input). The trust is not saved to `trustedFolders` and is not a tool or path permission grant. Cannot be combined with `--resume`, `--continue`, or `--session-id`.
`--sandbox`	Enable the OS-level shell sandbox for this session only, without changing your saved sandbox setting. Useful with `-p`.
`--no-sandbox`	Disable local sandboxing for this session only, without changing your saved sandbox setting. This option is ignored if an enterprise-managed policy has been configured to enforce sandboxing.
`--share=PATH`	Share a session to a Markdown file after completion of a programmatic session (default path: `./copilot-session-<ID>.md`).
`--share-gist`	Share a session to a secret GitHub gist after completion of a programmatic session.
`--stream=MODE`	Enable or disable streaming mode, which displays Copilot's response progressively as it is generated rather than waiting for the full response to arrive (mode choices: `on` or `off`, default: `on`).
`--sync-session[=on|off]`	Enable or disable read-only session history syncing for this run. Without a value, enables syncing. Does not enable remote control. If omitted, uses your settings and the default behavior. Syncing is refused when your organization's cloud session storage policy is off or can't be determined.
`-v`, `--version`	Show version information.

`-w`, `--worktree[=NAME]`	Create or reuse an isolated Git worktree under `<repo>.worktrees/` by default and start the session inside it. Use the `worktreePathTemplate` setting to configure the location. `NAME` is optional—omit it to auto-generate a branch name. By default, branches off the current checkout (`HEAD`); set the `worktreeBaseRef` setting to `"defaultBranch"` to branch off the remote default branch instead. Conflicts with `--resume` , `--continue` , and `--connect` .
`--yolo`	Enable all permissions (equivalent to `--allow-all`).

For a complete list of commands and options, run `copilot help` .

> **Note**
>
> Remote session features require availability on your account. The legacy `--remote` option is not an interactive-session control, and `--no-remote` is no longer supported.
>
> To enable remote control during an interactive session, use `/remote on` . To end the remote connection, use `/remote off` .

If a session was still open when its CLI process went away, for example due to a crash or a machine restart, the next time you start `copilot` you're offered the option to restore it. You can review the sessions available to restore or start a fresh session instead. A restored session whose agent was mid-turn automatically resumes that work.

Plan-then-autopilot

Plan-then-autopilot lets a session start in plan mode and automatically continue into autopilot mode once the plan is ready, without waiting for a human to approve the transition. Enable it with `--plan --mode autopilot` , or with the `COPILOT_PLAN_THEN_AUTOPILOT` environment variable for harnesses that can only inject environment variables and not command-line options. If both are set, the explicit options take precedence and the CLI displays a warning that the environment variable was ignored.

Enterprise-managed sandbox floor

An enterprise-managed policy can enforce OS-level shell sandboxing as a minimum floor. In other words, even if you pass `--no-sandbox` , the policy can still force sandboxing on. This is a policy override, not a failure of the option itself. By contrast, `--sandbox` is unaffected because it only turns sandboxing on and never removes it. If the effective policy permits sandbox bypass, you can explicitly disable sandboxing for the rest of the current session while responding to an active bypass permission prompt.

When a managed floor forces sandboxing on with an effective `sandbox.allowBypass` of `true` (the default when a policy sets only `sandbox.enabled` to `true`), run `/sandbox disable` to opt out of the sandbox for the rest of the current session—no bypass prompt is required first. The opt-

out is session-only: nothing is saved to `settings.json`, a new session starts sandboxed again, and `/sandbox enable` restores sandboxing immediately without waiting for a new session. If `allowBypass` is `false`, `/sandbox disable` refuses with a message that sandboxing is enforced by the managed policy and can't be disabled.

When a managed policy overrides your setting, the CLI shows a warning in the interactive timeline (or on stderr when using `-p`) so it is clear that the behavior comes from policy enforcement rather than the option failing to work. Contact your administrator if you need the policy changed. Use `/sandbox status` or `/sandbox policy` to inspect the session's effective restrictions.

Adding the managed `sandbox.failIfUnavailable` setting set to `true`, alongside `sandbox.enabled` set to `true`, makes the sandbox mandatory when it cannot be established. Instead of falling back to running commands unsandboxed, Copilot blocks model and tool execution if the policy can't be validated, compiled, or enforced by a usable sandbox backend. See Enterprise managed settings.

The CLI also warns when a managed policy enables sandboxing in a session that you did not request, not only when it overrides `--no-sandbox`. This includes sessions where the policy arrives after startup, because server-managed settings are only available after login. The warning is omitted if your own settings or the `--sandbox` option already requested sandboxing, since the session state would then be expected.

If a device has a managed policy that could not be read, the CLI fails closed and enforces sandboxing at its most restrictive level. The notice explains that the policy couldn't be determined and tells you to wait for it to resolve. The startup warning for an unsupported host uses the same wording.

Restricting the allow-all options

When `permissions.disableBypassPermissionsMode` is set to `"disable"`, all of the command line options that allow all permissions (`--allow-all-tools`, `--allow-all-paths`, `--allow-all-urls`, `--allow-all`, `--yolo`) are suppressed at startup and cannot be used to grant elevated permissions. The CLI shows a policy warning when it suppresses these options. The `/permissions allow-all` slash command, and its aliases `/allow-all` and `/yolo`, are also suppressed. This setting does not turn off assisted permissions, which are controlled separately.

What `disable` turns off

When the value of `permissions.disableBypassPermissionsMode` is `disable`, the command line options that grant all permissions have no effect. These options are: `--allow-all-tools`, `--allow-all-paths`, `--allow-all-urls`, `--allow-all`, and `--yolo`.

The restriction also applies during an interactive session. Slash commands that would turn on the allow-all mode are still available but are refused with an explanatory message.

Command	Result with `disable`
`/permissions allow-all`	Refused

`/allow-all` (and its alias `/yolo`) with no arguments, or `on`	Refused
`/permissions assisted`	Allowed
`/allow-all off`, `/allow-all show`, `/permissions default`, `/permissions show`	Allowed

When an allow-all command is refused, the message depends on where the restriction comes from. A restriction from your own settings reports "Bypass permissions mode has been disabled in your settings." A restriction from managed or MDM policy reports "Bypass permissions mode has been disabled by policy. Contact your administrator for more information."

Unrecognized values

If `permissions.disableBypassPermissionsMode` is set to a value this version does not recognize, the CLI does not reject it. Instead it logs the value and enforces `disable` as a safe default. A malformed policy still restricts the allow-all options rather than silently allowing them.

Where the setting can come from

Three sources can set `permissions.disableBypassPermissionsMode`. They are listed in increasing order of permanence.

Source	Scope	Cleared by account switch?
User settings (`~/.copilot/settings.json`)	Machine	No. It applies to all accounts.
Managed settings (server-fetched per account)	Account	Yes. The CLI loads the policy for the newly active account.
MDM policy (plist/registry/file)	Device	Never. Device-level policy is not overridden by account switches.

How the restriction behaves when the policy cannot be confirmed

A failed policy check does not necessarily leave the session unrestricted. The CLI distinguishes an unmanaged account or device from one whose applicable managed policy cannot be confirmed.

- **Unconfirmed managed policy can restrict permissions.** While policy checks are pending, or when policy cannot be confirmed for a managed account or device, the CLI can block permission escalation and require explicit approval for actions. This can apply even if no earlier policy disabled the allow-all options.
- **Server policy is account-specific.** If a refresh fails, previously retrieved policy for the same account can remain in effect. Switching accounts loads the new account's policy rather than carrying over the previous account's server policy.

- **Device restrictions persist across accounts.** Switching accounts does not remove MDM restrictions or user settings that block permission modes. A server response that omits the setting does not override those restrictions.

Supported models

Use `--model=MODEL` or the `COPILOT_MODEL` environment variable to select the AI model. Pass `auto` to let Copilot pick the best available model automatically.

Model	Best for
`claude-sonnet-5.5`	General-purpose coding (default)
`gpt-6.1-sol`	Recommended, opt-in (not the automatic default)
`gpt-6.1-terra`	Recommended, opt-in (not the automatic default)
`gpt-6-astra`	Recommended, opt-in (not the automatic default)
`gpt-6-luna`	Recommended, opt-in (not the automatic default)
`claude-opus-5.5`	Recommended, high-capability complex tasks
`claude-haiku-5.5`	Fast, lightweight operations
`claude-haiku-4.5`	Fast, lightweight operations
`gpt-5.3-codex`	Code-focused tasks
`gemini-3.8-flash`	Fast Google Gemini responses

You can also switch models during an interactive session using the `/model` slash command.

Tool availability values

The `--available-tools` and `--excluded-tools` options support these values:

Shell tools

Tool name	Description
`bash` / `powershell`	Execute commands
`list_bash` / `list_powershell`	List active shell sessions

`read_bash` / `read_powershell`	Read output from a shell session
`stop_bash` / `stop_powershell`	Terminate a shell session
`write_bash` / `write_powershell`	Send input to a shell session

File operation tools

Tool name	Description
`apply_patch`	Apply patches (used by some models instead of `edit` / `create`)
`create`	Create new files
`edit`	Edit files via string replacement
`view`	Read files or directories

Agent and task delegation tools

Tool name	Description
`list_agents`	List available agents
`read_agent`	Check background agent status
`task`	Run subagents
`write_agent`	Send a message to a running agent

Other tools

Tool name	Description
`ask_user`	Ask the user a question
`glob`	Find files matching patterns
`grep` (or `rg`)	Search for text in files
`skill`	Invoke custom skills
`web_fetch`	Fetch and parse web content

Tool permission patterns

The `--allow-tool` and `--deny-tool` options accept permission patterns in the format `Kind(argument)`. The argument is optional—omitting it matches all tools of that kind.

Kind	Description	Example patterns
`memory`	Storing facts to agent memory. See About GitHub Copilot Memory.	`memory`
`read`	File or directory reads	`read`, `read(.env)`
`shell`	Shell command execution	`shell(git push)`, `shell(git:*)`, `shell`
`url`	URL access via web-fetch or shell	`url(github.com)`, `url(https://*.api.com)`
`write`	File creation or modification	`write`, `write(src/*.ts)`
SERVER-NAME	MCP server tool invocation	`MyMCP(create_issue)`, `MyMCP`

For `shell` rules, the `:*` suffix matches the command stem followed by a space, preventing partial matches. For example, `shell(git:*)` matches `git push` and `git pull` but does not match `gitea`.

Deny rules always take precedence over allow rules, even when `--allow-all` is set.

```
# Allow all git commands except git push
copilot --allow-tool='shell(git:*)' --deny-tool='shell(git push)'

# Allow a specific MCP server tool
copilot --allow-tool='MyMCP(create_issue)'

# Allow all tools from a server
copilot --allow-tool='MyMCP'

# Deny writes to a specific path (exact or trailing-path-segment match; no glob
support yet)
copilot --deny-tool='write(secret.txt)'
```

`--deny-tool='write(PATH)'` scopes the denial to that path—other writes are unaffected. The match resolves symlinks and `.` / `..` segments, and is case-insensitive on macOS and Windows.

Environment variables

Variable	Description

`COPILOT_ALLOW_ALL`	Allow all permissions automatically (equivalent to `--allow-all`). Accepts `true`, `1`, `yes`, `on`, or `y` (case-insensitive, surrounding whitespace ignored) to turn it on; `false`, `0`, `no`, `off`, `n`, or an empty value to turn it off. Setting it to exactly `true` also trusts the working directory without prompting, loading that directory's skills, plugins, MCP servers, and hooks (including hooks that run shell commands). The other truthy spellings only auto-approve tools.
`COPILOT_ASSISTED_APPROVAL`	Set to `true` to turn on assisted permissions for a non-interactive run (equivalent to `--assisted-approval`).
`COPILOT_AUTO_UPDATE`	Set to `false` to disable automatic updates of the CLI and first-party plugins, and prevent a newer cached CLI version from running instead of the installed version.
`COPILOT_CACHE_HOME`	Override the cache directory (used for marketplace caches, auto-update packages, and other ephemeral data). See GitHub Copilot CLI configuration directory for platform defaults.
`COPILOT_CHILD_OOM_SCORE_ADJ`	Linux only. Overrides the `oom_score_adj` bias applied to shell command process trees spawned by the CLI, so the kernel's out-of-memory (OOM) killer reclaims a runaway child process tree (for example, a build) before the CLI itself. Default: `300`. Range: `-1000` – `1000`. Set to `off` to disable the bias.
`COPILOT_CUSTOM_INSTRUCTIONS_DIRS`	Comma-separated list of additional directories for custom instructions.
`COPILOT_EDITOR`	Editor command for interactive editing (checked after `$VISUAL` and `$EDITOR`). Defaults to `vi` if none are set.
`COPILOT_ENABLE_HTTP2`	Set to `1` or `true` to opt into HTTP/2 transport. HTTP/1.1 is the default.
`COPILOT_ENABLE_INTERRUPTED_SESSION_RESTORE`	Set to `1` to enable restoring a session that was still open when the CLI process went away, for example due to a crash or a machine restart. This restore behavior is temporarily opt-in while its startup dialog is reworked. See Restoring an interrupted session.

`COPILOT_GH_HOST`	GitHub hostname for Copilot CLI only, overriding `GH_HOST`. Use when `GH_HOST` targets GitHub Enterprise Server but Copilot needs to authenticate against GitHub.com or a GitHub Enterprise Cloud hostname.
`COPILOT_GITHUB_TOKEN`	Authentication token. Takes precedence over `GH_TOKEN` and `GITHUB_TOKEN`.
`COPILOT_HOME`	Override the configuration and state directory. Default: `$HOME/.copilot`.
`COPILOT_LARGE_OUTPUT_THRESHOLD_BYTES`	Maximum UTF-8 byte size for tool output returned directly to the model. Default: `20480` (20 KiB). See Managing context for GitHub Copilot.
`COPILOT_MCP_TOOL_CACHE`	Set to `false` to disable loading and persisting local MCP server tool snapshots for the entire process. See Tool snapshot caching.
`COPILOT_MODEL`	Set the AI model.
`COPILOT_PLAN_THEN_AUTOPILOT`	Set to `1`, `true`, `yes`, or `on` to request plan-then-autopilot (equivalent to `--plan --mode autopilot`) for harnesses that can only inject environment variables. Ignored, with a warning, when an explicit `--mode`, `--autopilot`, or `--plan` option is passed. See Plan-then-autopilot.
`COPILOT_PROMPT_FRAME`	Set to `1` to enable the decorative UI frame around the input prompt, or `0` to disable it. Overrides the `PROMPT_FRAME` experimental feature flag for the current session.
`COPILOT_PROVIDER_API_KEY_COMMAND`	Shell command that prints a fresh bring-your-own-key (BYOK) API key before each provider request. Takes precedence over the `COPILOT_PROVIDER_API_KEY` environment variable. Its output replaces the matching credential header even if a custom header set via `COPILOT_PROVIDER_HEADERS` would otherwise collide.
`COPILOT_PROVIDERS_CONFIG`	Path to a JSON file that defines a bring-your-own-key (BYOK) provider and model registry, as an object with `providers` and `models` keys. Default: `<COPILOT_HOME>/providers.json`. When this file declares any provider or model, it takes precedence over the legacy `COPILOT_PROVIDER_*` environment variables. See `providers.json`.
`COPILOT_SKILLS_DIRS`	Comma-separated list of additional directories for skills.

`PLUGINS_DASHBOARD`	Set to `false` to disable the plugins dashboard opened by bare `/plugin`, `/mcp`, and `/skills`, and to disable the non-interactive `copilot plugin` / `copilot plugins` commands.
`COPILOT_STRIP_REASONING_ON_RESUME`	Set to `0` or `false` to keep BYOK reasoning tokens across a session resume instead of stripping them. Defaults to stripping them.
`COPILOT_SUBAGENT_MAX_CONCURRENT`	Maximum concurrent subagents per session (default: `32`, range: `1` – `256`).
`COPILOT_SUBAGENT_MAX_DEPTH`	Maximum subagent nesting depth (default: `4`, range: `1` – `128`).
`COPILOT_TASK_WAIT_TIMEOUT_SECONDS`	Maximum seconds `-p` (and `-p --autopilot`) waits for pending background agents or shell commands to finish before exiting (default: `600`; `0` exits immediately without waiting).
`GH_HOST`	GitHub hostname for both GitHub CLI and Copilot CLI (default: `github.com`). Set to your GitHub Enterprise Cloud with data residency hostname. Override with `COPILOT_GH_HOST` for Copilot CLI only.
`GH_TOKEN`	Authentication token. Takes precedence over `GITHUB_TOKEN`.
`GITHUB_COPILOT_PROMPT_MODE_EXTENSIONS`	Set to `true` to load project extensions and allow extension management tools in prompt mode (`-p`). Disabled by default to prevent running repository-controlled extension code without interactive trust.
`GITHUB_COPILOT_PROMPT_MODE_REPO_HOOKS`	Set to `true` to load repository hooks in prompt mode (`-p`). Repository hooks also load automatically when the folder is already trusted or `COPILOT_ALLOW_ALL` is set.
`GITHUB_COPILOT_PROMPT_MODE_WORKSPACE_MCP`	Set to `true` to load workspace MCP sources in prompt mode (`-p`). Disabled by default to prevent starting repository-controlled MCP servers without interactive trust.
`GITHUB_TOKEN`	Authentication token.
`PLAIN_DIFF`	Set to `true` to disable rich diff rendering.
`USE_BUILTIN_RIPGREP`	Set to `false` to use the system ripgrep instead of the bundled version.
`USE_TGREP`	Set to `true` to always use tgrep, a trigram-indexed search engine, even outside a Git repository or on a

	virtualized or network filesystem (for example, a VFS for Git checkout, or an SMB/9p mount), or `false` to always use ripgrep. When forced outside a Git repository, tgrep indexes the working directory itself, so avoid forcing it somewhere large such as your home directory. Windows cloud-sync folders (for example, OneDrive) always use ripgrep, even when this is set to `true`. When unset, Copilot CLI automatically switches from ripgrep to tgrep only inside a Git repository, on a non-virtualized filesystem, and once it reaches the file-count threshold set by `TGREP_FILE_COUNT_THRESHOLD`. Setting `USE_TGREP=true` bypasses this threshold entirely. `USE_TGREP=false` still disables tgrep.
`TGREP_FILE_COUNT_THRESHOLD`	Minimum file count for automatic tgrep indexed search (non-negative integer; default: `10000` on Windows, `50000` elsewhere). Invalid values fall back to the platform default. `0` removes only the file-count requirement—other automatic-start checks still apply.

Configuration file settings

For detailed information about configuration file settings—including the full list of user settings, repository settings, local settings, and how they cascade—see GitHub Copilot CLI configuration directory.

Project initialization for Copilot

When you use the command `copilot init`, or the slash command `/init` within an interactive session, Copilot analyzes your codebase and writes or updates a `.github/copilot-instructions.md` file in the repository. This custom instructions file contains project-specific guidance that will improve future CLI sessions.

You will typically use `copilot init`, or `/init`, when you start a new project, or when you start using Copilot CLI in an existing repository.

The `copilot-instructions.md` file that's created or updated typically documents:

- Build, test, and lint commands.
- High-level architecture.
- Codebase-specific conventions.

If the file already exists, Copilot suggests improvements which you can choose to apply or reject.

The CLI looks for the `copilot-instructions.md` file on startup, and if it's missing, it displays the message:

> 💡 No copilot instructions found. Run /init to generate a copilot-instructions.md file for this project.

If you don't want to create this file, you can permanently hide this startup message for the current repository by using the `/init suppress` slash command.

For more information, see Adding repository custom instructions for GitHub Copilot.

Use `--sandbox` or `--no-sandbox` with `copilot init` to enable or disable the OS-level shell sandbox for the initialization session only, without changing your saved sandbox setting. These options are subject to the same enterprise-managed sandbox floor as other entry points—see Enterprise-managed sandbox floor.

Custom instructions locations

Copilot CLI loads custom instructions from these locations simultaneously (all are merged):

Location	Notes
`CLAUDE.md`	In Git root and cwd
`GEMINI.md`	In Git root and cwd
`AGENTS.md`	In Git root and cwd
`.github/instructions/**/*.instructions.md`	In Git root and cwd
`.claude/rules/**/*.md`	In Git root and cwd
`.github/copilot-instructions.md`	In Git root and cwd
`$HOME/.copilot/copilot-instructions.md`	–
`$HOME/.copilot/instructions/**/*.instructions.md`	–
`COPILOT_CUSTOM_INSTRUCTIONS_DIRS`	Additional directories via environment variable.

Copilot CLI reads Claude Code rule files from `.claude/rules/**/*.md` the same way it reads `.github/instructions/**/*.instructions.md` files. Scope a rule to matching files with a `paths` (or `applyTo`) frontmatter key:

```
---
paths: "src/api/**/*.ts"
description: API layer conventions
---

Use the shared `ApiError` type for all thrown errors in this directory.
```

Scoped rules at the repository root and in the working directory are delivered automatically when the agent reads or edits a matching file.

Custom instructions imports

Instruction files support `@path` imports. Prefix a line with `@` followed by a path to inline the contents of another file. Paths can be relative to the instruction file's directory or absolute. Imports are resolved recursively, up to a depth limit, with cycle and size guards. This is supported in `AGENTS.md`, `CLAUDE.md`, and `.github/copilot-instructions.md`.

Hooks reference

For detailed information about hooks—including hook configuration formats, hook events, input payloads, and decision control—see GitHub Copilot hooks reference.

MCP server configuration

MCP servers provide additional tools to the CLI agent. Configure persistent servers in `~/.copilot/mcp-config.json`. Use `--additional-mcp-config` to add servers for a single session.

Local (stdio) servers that spawn inside the sandbox (see the `/sandbox` slash command) show a `connected (sandboxed)` status in `copilot mcp list` and `/mcp list`. Remote (HTTP/SSE) servers are not locally sandboxed, although their connections from the CLI can be restricted by the sandbox's network policy.

`copilot mcp list` and `/mcp list` mark a disabled server with a `(disabled)` suffix in text output, or `"enabled": false` per server in `--json` output. `copilot mcp get` shows a `Status: Enabled` / `Disabled` line.

Toggling `/sandbox` restarts only local (stdio) MCP servers, since they spawn inside the sandbox. Remote (HTTP/SSE) servers stay connected.

`/mcp edit <name>` rejects a workspace-sourced server—one defined in a repository's `.mcp.json`—instead of opening the user-tier wizard, since saving would silently create a same-name user entry that the workspace one still shadows. The error names the file to edit directly. `/mcp delete <name>` reports the same file when asked to remove a workspace-sourced server.

`copilot mcp` subcommand

Use `copilot mcp` to manage MCP server configurations from the command line without starting an interactive session.

Subcommand	Description

`list [--json]`	List all configured MCP servers grouped by source, including plugin-provided servers and servers supplied through `--additional-mcp-config` or `--plugin-dir`.
`get <name> [--json]`	Show configuration and tools for a specific server. For plugin-provided servers, also shows the source plugin name and version.
`add [options] <name> [url]`	Add a server to the user configuration. Writes to `~/.copilot/mcp-config.json`.
`enable <name>` / `disable <name>`	Enable or disable a server by name. The change persists to the user configuration and applies to future sessions.
`remove <name>`	Remove a user-level server. Workspace servers must be edited in their configuration files directly.

For local (stdio) servers, provide the command after `--`:

Shell

```
copilot mcp add SERVER-NAME -- COMMAND [ARGS...]
```

For remote HTTP or SSE servers, specify the transport and provide the URL:

Shell

```
copilot mcp add --transport http SERVER-NAME URL
```

`copilot mcp add` options:

Option	Description
`-- <command> [args...]`	Command and arguments for local (stdio) servers.
`<url>`	URL for remote servers.
`--transport <transport>`	Transport type: `stdio`, `http`, or `sse`. The default is `stdio`.
`--env KEY=VALUE`	Environment variable (repeatable).
`--header "HEADER: VALUE"`	HTTP header for remote servers (repeatable).
`--tools <tools>`	Tool filter: `"*"` for all, a comma-separated list, or `""` for none.

`--timeout <ms>`	Timeout in milliseconds for tool discovery and tool calls. Default: `30000`. Must be a positive integer with no fractional part, unit suffix, sign, or exponent, from `1` to `4294967295`.
`--json`	Output added configuration as JSON.
`--show-secrets`	Show full environment variable and header values.

In the `/mcp add` / `/mcp edit` interactive form, enter the `env` field as comma-separated `KEY=VALUE` pairs or as a JSON object (for example, `{"API_KEY":"secret"}`). `$PATH` is included by default and doesn't need to be listed.

> **Caution**
>
> `--show-secrets` can print sensitive environment variable and header values to your terminal or logs. Only use this option in trusted environments, and avoid copying, pasting, or otherwise capturing the output in shared logs or history.

Transport types

Type	Description	Required fields
`local` / `stdio`	Local process communicating via stdin/stdout.	`command`, `args`
`http`	Remote server using streamable HTTP transport. `"streamable-http"` is also accepted as an alias and is normalized to `"http"`.	`url`
`sse`	Remote server using Server-Sent Events transport.	`url`

Local server configuration fields

Field	Required	Description
`command`	Yes	Command to start the server.
`args`	Yes	Command arguments (array).
`tools`	Yes	Tools to enable: `["*"]` for all, or a list of specific tool names.
`env`	No	Environment variables. Supports `$VAR`, `${VAR}`, and `${VAR:-default}` expansion.

`cwd`	No	Working directory for the server.
`timeout`	No	Timeout in milliseconds for tool discovery and tool calls. Default: `30000`.
`type`	No	`"local"` or `"stdio"`. Default: `"local"`.
`deferTools`	No	`"auto"` (default) or `"never"`. Set to `"never"` to keep this server's tools always visible even when tool search is active.
`disableToolCache`	No	Set to `true` to skip loading and persisting the tool snapshot for this server.
`slowConnectionThresholdMs`	No	Milliseconds before Copilot CLI warns that this server is taking longer than expected to connect. Default: `10000`. This setting only affects when the warning is shown—it doesn't change the connection timeout. Must be a positive integer no greater than `2147483647`.

`slowConnectionThresholdMs` and `timeout` are independent: `timeout` sets the actual connection budget (floored at `60000` ms), while `slowConnectionThresholdMs` only controls when Copilot CLI prints a warning for a connection that's still in progress. Raise `slowConnectionThresholdMs` to quiet expected-slow handshakes, for example for servers behind an authentication flow, without extending how long tool calls are allowed to take.

Tool snapshot caching

Copilot CLI persists a snapshot of each local server's tool list so tools are available immediately on startup while live discovery completes in the background. Live discovery always runs and replaces the snapshot once it finishes.

Set `disableToolCache: true` on a server to force live discovery for that server only, or set the `COPILOT_MCP_TOOL_CACHE=false` environment variable to disable snapshot loading and persistence for the entire process. Both opt-outs leave existing cache files untouched.

Private npm registry

Use `--registry` in the `args` array to pull a package from a private npm registry—for example, Artifactory or a GitHub Packages feed:

```
{
    "mcpServers": {
        "my-internal-server": {
            "command": "npx",
            "args": [
                "--registry", "https://npm.pkg.github.com",
                "@my-org/internal-mcp-server"
            ],
            "tools": ["*"]
        }
    }
}
```

The `--registry` option and other npm configuration options (`--userconfig` , `--globalconfig` , `--prefix` , `--cache` , `--node-options` , `--workspace` , `-w`) are treated as value-consuming arguments when computing the server's identity fingerprint. This ensures enterprise allowlist checks and registry verification work correctly when these options appear before the package name.

Remote server configuration fields

Field	Required	Description
`type`	Yes	`"http"` or `"sse"` (`"streamable-http"` is also accepted as an alias for `"http"`).
`url`	Yes	Server URL.
`tools`	Yes	Tools to enable.
`headers`	No	HTTP headers. Supports variable expansion.
`oauthClientId`	No	Static OAuth client ID (skips dynamic registration).
`oauthScopes`	No	Non-empty array of OAuth scope tokens to request. Requires `oauthClientId` . A non-empty scope in the server's `WWW-Authenticate` challenge still takes precedence; otherwise this overrides the discovered `scopes_supported` metadata.
`oauthPublicClient`	No	Whether the OAuth client is public. Default: `true` . Set to `false` for confidential clients with a stored secret.

`oauthGrantType`	No	OAuth grant type: `"authorization_code"` (default, browser-based flow) or `"client_credentials"` (fully headless, no browser or callback).
`oidc`	No	Enable OIDC token injection. When `true`, the CLI injects OIDC tokens for any `GITHUB_COPILOT_OIDC_MCP_TOKEN` or `GITHUB_COPILOT_OIDC_MCP_TOKEN_<SUFFIX>` variable referenced in the server's `env` block (local servers), or sends the token as a `Bearer` `Authorization` header (remote servers). For local servers, prefer suffixed variants (for example, `${GITHUB_COPILOT_OIDC_MCP_TOKEN_MY_SVC}`) to assign a unique variable name per server.
`timeout`	No	Timeout in milliseconds for tool discovery and tool calls. Default: `30000`.
`deferTools`	No	`"auto"` (default) or `"never"`. Set to `"never"` to keep this server's tools always visible even when tool search is active.
`slowConnectionThresholdMs`	No	Milliseconds before Copilot CLI warns that this server is taking longer than expected to connect. Default: `10000`. This setting only affects when the warning is shown –it doesn't change the connection timeout. Must be a positive integer no greater than `2147483647`.

OAuth re-authentication

Remote MCP servers that use OAuth may show a `needs-auth` status when a token expires or when a different account is required. Use `/mcp auth <server-name>` to trigger a fresh OAuth flow. This opens a browser authentication prompt, allowing you to sign in or switch accounts. After completing the flow, the server reconnects automatically.

On Windows, remote MCP servers protected by Microsoft Entra ID authenticate through the OS authentication broker (Web Account Manager) instead, usually with no prompt. On Linux and

macOS, where no OS broker exists, the CLI instead silently mints a per-resource token from the Entra account established via `/login` → Microsoft Entra. If that account can't silently satisfy the request—for example, no signed-in Entra identity, an ambiguous account, or a resource pre-authorized for a different client—sign-in falls back to the browser flow described above. Passing `--device-code` bypasses the broker and forces the OAuth device code flow rather than the browser flow, and Windows machines without the broker library also fall back to the browser flow.

Account-derived WorkIQ

Signing in with a Microsoft Entra account makes the hosted WorkIQ MCP server available in `/mcp`. It's disabled by default. Discovery alone doesn't connect, acquire a token, or expose tools—enable it with the normal MCP controls (`/mcp enable WorkIQ` or `copilot mcp enable WorkIQ`) to try silent authentication first. `/mcp auth WorkIQ` remains available for recovery.

The server uses a credential tied to the Entra account that signed in, not a token issued for another service, and its enablement follows that Entra identity rather than transferring to a different account. Explicit server configuration and disablement, and organization allowlist or denylist policy, still take precedence. No WorkIQ entry is written to `mcp-config.json`. This is unrelated to running `npx @microsoft/workiq mcp` as an independently authenticated stdio server.

Headless OAuth (`client_credentials` grant)

For CI or cron use cases where no browser is available, set `oauthGrantType: "client_credentials"`. This requires:

- `oauthClientId`—the static client ID issued by the MCP provider.
- `oauthPublicClient: false`—the client is confidential.
- A `client_secret` stored in the system keychain (configured once via the `/mcp` UI or by writing to the OAuth credential store).

When configured, the CLI skips the browser, callback server, PKCE, and dynamic client registration entirely. On every 401, it posts `grant_type=client_credentials` directly to the server's discovered token endpoint.

```
{
    "mcpServers": {
        "headless-api": {
            "type": "http",
            "url": "https://api.example.com/mcp",
            "tools": ["*"],
            "oauthClientId": "YOUR-CLIENT-ID",
            "oauthPublicClient": false,
            "oauthGrantType": "client_credentials"
        }
    }
}
```

Filter mapping

Control how MCP tool output is processed using the `filterMapping` field in a server's configuration.

Mode	Description
`none`	No filtering.
`markdown`	Format output as Markdown.
`hidden_characters`	Remove hidden or control characters. Default.

Built-in MCP servers

The CLI automatically configures `github-mcp-server` when authentication is available, unless you disable it or supply your own configuration. Other servers depend on your account, platform, CLI version, and enabled plugins. They are not all available or enabled by default.

Server	Description	Availability and setup
`github-mcp-server`	GitHub integration, including repository access and code search.	Enabled by default when authentication is available. The exposed tools depend on your configuration.
`WorkIQ`	Hosted MCP server associated with a Microsoft Entra account.	Discovered after you sign in with a Microsoft Entra account. Disabled by default. See Account-derived WorkIQ.
`browser-use`	Browser interaction through a bundled plugin.	Available in CLI versions that include the plugin. Disabled by default. Enable it with `/mcp enable browser-use`, install the matching browser extension and Node.js 22 or later, then run `/browser-use:activate` or start a new session. The legacy server name is `github-copilot-browser`.
`computer-use`	Desktop application interaction through a bundled plugin.	Available for local sessions on macOS and Windows. Disabled by default. Enable it with `/computer on`. On macOS, Accessibility and Screen Recording permissions are required. See Using GitHub Copilot CLI to interact with desktop applications.

Managed settings can prevent you from enabling a server or its plugin.

`--disable-builtin-mcps` disables `github-mcp-server`, `WorkIQ`, `browser-use`, and the legacy browser server name `github-copilot-browser` for the current run. It does not disable `computer-use`. Use `--disable-mcp-server computer-use` to disable that server for the current run, or `/computer off` to disable the plugin and save that preference. Use `--disable-mcp-server SERVER-NAME` to disable any other specific server for the current run.

Use `/mcp list` in an interactive session to check which servers are available and their connection status. Use `copilot mcp list` or `copilot mcp get SERVER-NAME` outside a session to inspect server configuration. A server that is absent or disabled is not necessarily a configuration error. Check its availability and setup requirements above.

GitHub MCP server tools

The `github-mcp-server` provides the following tools.

Tools	Description
`get_file_contents`, `search_code`	Browse repository files.
`list_issues`, `issue_read`, `search_issues`	Issue tracking.
`get_pull_request`, `list_pull_requests`, `get_pull_request_files`	Pull requests.
`list_commits`, `get_commit`	Commit history.
`list_workflow_runs`, `get_workflow_run_logs`	GitHub Actions.
`get_label`, `list_label`, `label_write`	Label management.

Resource discovery

Only available in experimental mode. When a local capability is missing, the agent can search a remote catalog for public MCP servers and skills to add, using a built-in `discover-resources` skill. The skill queries the catalog for relevance-ranked MCP servers and skills matching an abstract capability (for example, "database access" or "architecture diagrams"), then presents candidates for you to choose from. The skill itself never installs anything. The `/plugin`, `/mcp`, and `/skills` dashboards also include a typed catalog search that blends the same remote results into the local resource list.

MCP server naming

Server names can contain any printable characters, including spaces, Unicode characters, and punctuation. Control characters (U+0000–U+001F, U+007F) and the closing brace (`}`) are not allowed. Server names are used as prefixes for tool names—for example, a server named `my-`

`server` produces tool names like `my-server-fetch`, and a server named `My Server` produces `My Server-fetch`.

MCP tool name sanitization

MCP server names and tool names are sanitized before being sent to the model. Characters that are invalid in tool names (anything other than `a-z`, `A-Z`, `0-9`, `-`, `_`) are replaced with `-`. Unicode characters are Punycode-encoded. The `@` symbol is also replaced with `-` to avoid conflicts with Punycode encoding.

The combined name (`serverName-toolName`) is capped at 64 characters. When truncation would create a name collision, a numeric suffix is appended (for example, `my-server-tool2`, `my-server-tool3`) to ensure uniqueness.

MCP server trust levels

MCP servers are loaded from multiple sources, each with a different trust level.

Source	Trust level	Review required
Built-in	High	No
Repository (`.github/mcp.json`)	Medium	Recommended
Workspace (`.mcp.json`)	Medium	Recommended
User config (`~/.copilot/mcp-config.json`)	User-defined	User responsibility
Remote servers	Low	Always

MCP tool invocations are subject to the session's permission rules, including existing approvals and assisted permissions when enabled. Even read-only operations on external services can require approval.

MCP server loading priority

MCP servers from different sources are merged in priority order (highest first). When servers share a name, the higher-priority source takes precedence.

1. `--additional-mcp-config` option (highest)
2. Plugin-provided servers
3. Workspace servers—`.mcp.json` and `.github/mcp.json` loaded from the working directory upward to the Git root; requires the folder to be trusted
4. `~/.copilot/mcp-config.json` (lowest)

> **Note**
>
> Workspace MCP servers (`.mcp.json` and `.github/mcp.json`) are loaded in both interactive and SDK server-mode sessions, provided the working directory is trusted. For more information about folder trust, see Allowing and denying tool use.

If a workspace configuration file contains an invalid server entry, the CLI skips only that entry and keeps loading its valid siblings, printing `Warning: workspace MCP config "<path>": <message>` for each skipped entry. A malformed or unreadable file (invalid JSON or an invalid top-level structure) is still skipped entirely.

Enterprise MCP allowlist

GitHub Enterprise organizations can enforce an allowlist of permitted MCP servers. When active, the CLI evaluates each non-default server against the enterprise policy before connecting.

When a GitHub Enterprise registry policy is detected (or the `MCP_ENTERPRISE_ALLOWLIST` experimental feature flag is enabled), the CLI:

1. Computes a fingerprint for each configured non-default server based on its command, arguments, and remote URL.
2. Sends the fingerprints to the enterprise allowlist evaluate endpoint.
3. Allows only servers whose fingerprints are approved; all others are blocked with a message naming the enterprise.

This check is fail-closed: if the evaluate endpoint is unreachable or returns an error, non-default servers are blocked until the policy can be verified.

When a server is blocked by an enterprise allowlist, the CLI displays:

```
MCP server "SERVER-NAME" was blocked by your enterprise "ENTERPRISE-NAME".
Contact your enterprise administrator to add this server to the allowlist.
```

Built-in default servers are always exempt from allowlist enforcement.

Migrating from `.vscode/mcp.json`

If your project uses `.vscode/mcp.json` (VS Code's MCP configuration format), migrate to `.mcp.json` for GitHub Copilot CLI. The migration remaps the `servers` key to `mcpServers` .

POSIX shells (bash, zsh, fish, and others):

```
jq '{mcpServers: .servers}' .vscode/mcp.json > .mcp.json
```

Requires `jq` .

PowerShell:

```
pwsh -NoProfile -Command "`$json = Get-Content '.vscode/mcp.json' -Raw |
ConvertFrom-Json; `$content = ([pscustomobject]@{ mcpServers = `$json.servers } |
ConvertTo-Json -Depth 100); [System.IO.File]::WriteAllText('.mcp.json',
`$content, (New-Object System.Text.UTF8Encoding `$false))"
```

On Windows, replace `pwsh` with `powershell` if you are using Windows PowerShell instead of PowerShell Core.

Stdio server output

The MCP stdio transport reserves stdout exclusively for newline-delimited JSON-RPC frames. The CLI automatically filters out any non-JSON lines—plain-text logs, exception stack traces, or whitespace-only lines—before passing output to the protocol parser.

Write all diagnostic output to **stderr**, not stdout. A server that writes logs or error messages to stdout can trigger a parse-error feedback loop that stalls the initialization handshake; the filter prevents this by silently dropping non-JSON frames.

Lines longer than 1 MB bypass the structural check and are forwarded as-is, to avoid splitting or dropping oversized but valid protocol frames (for example, a large `tools/list` response).

Skills reference

Skills are Markdown files that extend what the CLI can do. Each skill lives in its own directory containing a `SKILL.md` file. When invoked (via `/SKILL-NAME` or automatically by the agent), the skill's content is injected into the conversation.

Skill frontmatter fields

Field	Type	Required	Description
`name`	string	Yes	Unique identifier for the skill. Must start with a letter or number and contain only letters, numbers, hyphens, underscores, dots, colons, and spaces. Max 64 characters. Colons allow namespaced names (for example, `my-plugin:search`).
`description`	string	Yes	What the skill does and when to use it. Max 1024 characters.

`argument-hint`	string	No	Freeform hint describing expected arguments, shown in the skill picker (for example, `"[target] [mode]"`).
`allowed-tools`	string or string[]	No	Comma-separated list or YAML array of tools that are automatically allowed when the skill is active. Use `"*"` for all tools.
`user-invocable`	boolean	No	Whether users can invoke the skill with `/SKILL-NAME`. Default: `true`.
`disable-model-invocation`	boolean	No	Prevent the agent from automatically invoking this skill. Default: `false`.

Skill locations

Skills are loaded from these locations in priority order (first found wins for duplicate names).

Location	Scope	Description
`.github/skills/`	Project	Project-specific skills.
`.agents/skills/`	Project	Alternative project location.
`.claude/skills/`	Project	Claude-compatible location.
Parent `.github/skills/`	Inherited	Monorepo parent directory support.
`~/.copilot/skills/`	Personal	Personal skills for all projects.
`~/.agents/skills/`	Personal	Agent skills shared across all projects.
Plugin directories	Plugin	Skills from installed plugins.
`COPILOT_SKILLS_DIRS`	Custom	Additional directories (comma-separated).
`--add-dir <path>`	Added root	`.github/skills/`, `.agents/skills/`, and `.claude/skills/` under directories added with `--add-`

		`dir`, `/add-dir`, or the SDK's `additionalDirectories`. This is a trust decision: added skills run with the same trust as project skills.
(bundled with CLI)	Built-in	Skills shipped with the CLI. Lowest priority—overridable by any other source.
(org/enterprise)	Remote	Skills hosted by your organization or enterprise, projected via the AHP relay. Content is fetched on demand when the skill is invoked.

Remote skills are projected alongside local skills and follow the same name-based priority when a local skill has the same name.

Use the `ignoredSkillsLocations` setting to exclude specific directories (and their descendants) from discovery, regardless of which location above would otherwise surface them. See GitHub Copilot CLI configuration directory.

When two plugins provide skills with the same name, both coexist using plugin-qualified invocation names such as `/my-plugin/search` and `/other-plugin/search`. The bare name routes to the higher-priority plugin. This applies to skills only; commands keep the standard tier-based deduplication, where the higher-priority source wins.

Managing skills non-interactively

Use `copilot skill` to manage skills from the command line without opening an interactive session.

Subcommand	Description
`list [--json]`	List all discovered skills.
`add <source> [--project]`	Add a skill from a file path, URL, or directory.
`remove <name-or-directory>`	Remove a personal or project skill, or unregister a custom skill directory.
`enable <name>` / `disable <name>`	Enable or disable a skill by name.

```
# Install a skill for your user account (default)
copilot skill add ./my-skill/SKILL.md

# Install a skill into the current project (.github/skills; file or URL skills
only)
copilot skill add --project ./my-skill/SKILL.md

# Enable, disable, or remove a skill by name
```

```
copilot skill enable my-skill
copilot skill disable my-skill
copilot skill remove my-skill
```

Installing a directory registers it as a custom skill source rather than copying it. Installing a file or URL copies the skill's content into your personal or project skills directory. The equivalent interactive command is `/skills add [--project] <FILE|URL|DIRECTORY>`.

`copilot skill list --json` rows have the shape `{ name, description, source, path, enabled }`. With `remove`, pass either a skill name or the path to a custom skill directory you added—a skill name deletes that skill's files, while a custom directory path only unregisters the directory and leaves its files on disk. Only personal and project skills you added can be removed; skills provided by a plugin or the builtin set can't be removed this way (disable them instead). This replaces the retired `copilot plugins install --skill [--scope project]` and `copilot plugins remove/enable/disable --skill`.

Commands (alternative skill format)

Commands are an alternative to skills stored as individual `.md` files in `.claude/commands/`. The command name is derived from the filename. Command files use a simplified format (no `name` field required) and support `argument-hint`, `description`, `allowed-tools`, and `disable-model-invocation`. Commands have lower priority than skills with the same name.

Extension modes

> **Note**
>
> GitHub Copilot CLI extensions are currently an experimental feature and are subject to change.

In an interactive session, `/extensions mode` controls whether extensions run and whether Copilot can manage them.

Mode	Extensions run	Agent can manage extensions
Load & Augment (default)	Yes	Yes
Load Only	Yes	No
Disabled	No	No

In **Load & Augment** mode, the agent has tools to list discovered extensions, inspect their status and logs, scaffold a starter extension, and reload extensions after code changes. This lets the agent create and load extension code as part of your request.

Choose **Load Only** to run an existing set of trusted extensions without giving the agent extension-management tools. The agent can still call tools supplied by running extensions.

Mode changes take effect immediately in the current session:

- Switching to **Disabled** stops running extension processes and removes their tools.
- Switching from **Disabled** to either loading mode starts extensions.
- Switching between **Load Only** and **Load & Augment** changes access to extension-management tools without stopping the extensions.

Disabling extensions persists for future sessions until you choose a loading mode. Other sessions that are already running are not affected.

To inspect extensions and their status, use `/extensions manage`. For discovery locations and implementation examples, see Creating extensions for GitHub Copilot CLI.

Custom agents reference

Custom agents are specialized AI agents defined in Markdown files. For user, project, and added-root agents, the agent ID is derived from the file's path relative to the `agents` directory. The CLI removes the `.agent.md` or `.md` extension and replaces directory separators with `--`. For example, `agents/team/reviewer.agent.md` has the ID `team--reviewer`.

Built-in agents

Agent	Default model	Description
`code-review`	claude-sonnet-4.6	High signal-to-noise code review. Analyzes diffs for bugs, security issues, and logic errors. Will not modify code.
`explore`	gpt-5.6-luna	Fast codebase exploration. Searches files, reads code, and answers questions. Returns focused answers under 300 words. Safe to run in parallel.
`general-purpose`	claude-sonnet-4.6	Full-capability agent for complex multi-step tasks. Runs in a separate context window.
`research`	claude-haiku-4.5	Executes thorough searches based on instructions. Searches GitHub repositories, fetches files, verifies claims, and reports detailed findings with citations.
`rubber-duck`	complementary model	Use a complementary model to provide a constructive critique of proposals, designs,

		implementations, or tests. Identifies weak points and suggests improvements. See Getting a second opinion with the rubber duck agent.
`security-review`	claude-sonnet-4.6	Security-focused code review. Analyzes changes for high-confidence vulnerabilities across 11 categories. Only flags issues with >80% confidence of exploitability. Reports severity and confidence scores. Will not modify code.
`task`	gpt-5.6-luna (falls back to claude-haiku-4.5)	Command execution (tests, builds, lints). Returns brief summary on success, full output on failure.

`code-review` and `security-review` never forward a full review to another review agent: launching one already fulfills a request to "use" or "call" a reviewer, so it performs the review itself instead of delegating the whole task to a nested `code-review` or `security-review` subagent, recursively, across the entire delegation chain. `code-review` still hands off to the dedicated `security-review` specialist for security-focused portions of a request, and both agents may delegate narrow, independently scoped fact-finding work to non-review agents such as `explore`.

Only the root agent can call `store_memory` or `vote_memory` to save or vote on a memory. Subagents keep read access to stored memories through `read_memories`, but can't write or vote on them.

Custom agent frontmatter fields

Field	Type	Required	Description
`description`	string	Yes	Description shown in the agent list and `task` tool.
`include-custom-instructions`	boolean	No	Include repository instruction files (`copilot-instructions.md`, `AGENTS.md`, `CLAUDE.md`) when this agent runs as a subagent. Default: `false`. See Repository custom instructions for subagents.
`infer`	boolean	No	Allow auto-delegation by the main agent. Default:

			`true`.
`mcp-servers`	object	No	MCP servers to connect. Uses the same schema as `~/.copilot/mcp-config.json`.
`model`	string	No	AI model for this agent. When unset, inherits the outer agent's model. When the session model is set to `Auto` (server-selected), subagents always inherit the resolved session model regardless of this field.
`models`	string[]	No	Authored models in priority order. The runtime uses the first model the user's plan can access; if none resolve, dispatch falls back to the session's model. Overrides `model` when both are set.
`modelPolicy`	string	No	`"preferred"` (default) lets `model` / `models` be overridden by a `subagents` override in `~/.copilot/settings.json` or the `/subagents` picker. `"required"` locks dispatch to one of the authored models—overrides are rejected and the picker's **Model** entry is disabled.
`name`	string	No	Display name. You can also use this value to select the agent, for example with `--agent`. Defaults to the agent ID.
`reasoningEffort`	string	No	Default reasoning effort for this agent (for example, `"low"`, `"medium"`, or `"high"`). When unset, inherits the outer agent's effort.

`tools`	string[]	No	Tools available to the agent. Default: `["*"]` (all tools). Include `*` anywhere in the list to grant full tool access–for example, `["view", "*"]` grants every tool, not just `view`.

`model`, `models`, `modelPolicy`, and `reasoningEffort` apply whether the agent is dispatched through the `task` tool or started directly–for example, through the SDK's `session.startSubagent`. Model and effort are resolved with this precedence, highest first: an explicit per-call value, the `subagents` override in `~/.copilot/settings.json`, the agent definition's `model` / `models` / `reasoningEffort` field, then the parent session's value. A declared model or effort that can't be honored falls back to the session's value instead of failing the dispatch–unless the agent declares `modelPolicy: "required"`, in which case dispatch is refused rather than silently substituting an unauthored model.

Custom agent locations

The CLI loads custom agents in the following priority order. When agents have the same ID, the first agent loaded is used.

Priority	Scope	Location
1 (highest)	User	`~/.copilot/agents/`
2	Project	`.github/agents/`, from the current working directory upward to the Git root
3	Project	`.claude/agents/`, from the current working directory upward to the Git root
4	Added root	`.github/agents/` and `.claude/agents/` under a directory added with `--add-dir`, `/add-dir`, or the SDK's `additionalDirectories`. Adding the directory is a trust decision: its agents are loaded as trusted configuration.
5	Plugin	`<plugin>/agents/`
6 (lowest)	Remote	Organization or enterprise agents

Within each project row, the CLI checks the current working directory first, then each parent directory up to the Git root, so the deepest directory has the highest priority. Every project

`.github/agents/` directory is loaded before any project `.claude/agents/` directory. This means each package or subdirectory in a monorepo can contribute its own agents.

The optional `name` frontmatter field does not control deduplication. Agents with the same `name` but different IDs both load. If the value passed to `--agent` matches more than one agent's ID or `name`, the CLI uses the first matching agent in the priority order above.

Repository custom instructions for subagents

A custom agent spawned as a subagent—through the `task` tool or the SDK's `session.startSubagent`—doesn't receive repository instruction files (`copilot-instructions.md`, `AGENTS.md`, `CLAUDE.md`) by default. Set `include-custom-instructions: true` in its frontmatter to opt in:

```
---
name: RepoAwareReviewer
description: Reviews a change against this repository's conventions
tools: ["*"]
include-custom-instructions: true
---

Review the change and flag anything that violates the repository's conventions.
```

Agent	Receives repository instructions
Session agent (the default agent, or a custom agent selected with `--agent`)	Yes
`general-purpose` subagent	Yes
Built-in `explore`, `task`, and `code-review` subagents	No
Custom agent spawned as a subagent	Only with `include-custom-instructions: true`

When a subagent has `include-custom-instructions: true`, it reads your repository's own instruction files (`copilot-instructions.md`, `AGENTS.md`, `CLAUDE.md`). It does not inherit any extra instruction directories you loaded only for the main session—for example, a personal directory you pointed the session at with `COPILOT_CUSTOM_INSTRUCTIONS_DIRS`. Starting a session with `--no-custom-instructions` always takes priority: when you use that option, no agent receives repository instructions, even one that sets `include-custom-instructions: true`. Only turn the setting on for agents that need to follow your repository's conventions, such as an agent that reviews or edits code. Repository instructions add to what an agent processes each time it runs, so leaving the setting off for agents that don't need them—for example, an agent that only searches for files or summarizes text—keeps those agents faster and cheaper to run.

For more information, see Creating and using custom agents for GitHub Copilot CLI.

Agent communication

Use `list_agents` and `write_agent` inside custom agents to inspect nearby agents and coordinate work in a multi-agent session.

Relation labels in `list_agents`

Relation labels identify how visible agents relate to the current agent. Labels appear when a subagent runs inside a parent session with shared sibling communication enabled.

Label	Meaning	Use it to
`"self"`	The current agent	Confirm which entry represents the active agent
`"sibling"`	An agent launched by the same parent	Coordinate with peer agents through `write_agent`
`"child"`	An agent launched by the current agent	Track follow-up work delegated from the current agent

Scoped listing

Use `scope` on `list_agents` to narrow the list before choosing targets.

`scope` value	Returns	Use it to
omitted	Nearby agents in the current context	See the default working set for the current workflow
`"siblings"`	Only sibling agents	Find peer agents launched by the same parent
`"children"`	Only child agents of the current agent	Review work delegated from the current agent
`"all"`	All visible agents	Inspect the full session tree without using it for coordination

In single-agent sessions, the default view centers on child agents. In multi-agent sessions, the default view shows the immediate local context instead of the entire tree.

```
list_agents(scope="siblings")
list_agents(scope="children")
list_agents(scope="all")
```

Scoped messaging

Use `scope` on `write_agent` to broadcast one message to multiple related agents.

Use scoped messaging only from a subagent running inside a parent session with shared sibling communication enabled. In top-level sessions, target agents with explicit `agent_id` values instead.

`scope` value	Sends to	Use it to
`"siblings"`	All visible sibling agents	Coordinate peer work in a shared session
`"children"`	All child agents of the current agent	Send the same follow-up to delegated work

If a scope matches too many agents, `write_agent` returns an error and asks for explicit `agent_id` values from `list_agents` instead.

```
write_agent(scope="children", message="Re-check your findings against the
updated schema.")
write_agent(scope="siblings", message="Post status when your current check
completes.")
write_agent(agent_id="explore-auth", message="Focus on token refresh flow and
report only confirmed issues.")
```

Subagent limits

The CLI enforces depth and concurrency limits to prevent runaway agent spawning.

Limit	Default	Max
Max depth	`6`	`256`
Max concurrent	plan-based	`32`

Depth counts how many agents are nested within one another. When the depth limit is reached, the innermost agent cannot spawn further subagents. **Concurrency** counts how many subagents are running simultaneously across the entire session tree. When the limit is reached, new subagent requests are rejected until an active agent completes.

The default concurrency limit depends on your Copilot plan:

Plan	Max concurrent
Free / Education	`2`
Pro / Pro+	`4`
Max	`8`
Business	`16`

Enterprise	`32`
Usage-based billing	`32`

Usage-based billing users can override these limits with the `subagents.maxConcurrency` and `subagents.maxDepth` settings:

```
{
    "subagents": {
        "maxConcurrency": 16,
        "maxDepth": 10
    }
}
```

Values outside the valid range are clamped: `maxConcurrency` is capped at `32`, and `maxDepth` is capped at `256`. These settings are ignored for plans that don't use usage-based billing. See Configuration file settings.

Sidekick agents

Sidekick agents run automatically in the background and publish context into the session inbox. They respond to session events rather than being explicitly invoked.

Add a `sidekick:` block to any agent definition to make it a sidekick agent:

```
---
name: Context Gatherer
description: Gathers relevant context when the working directory changes
sidekick:
    triggers:
        - session.context_changed
        - event: user.message
          limit: 1
    behavior: persistent
    maxSendsPerTurn: 2
---

Gather useful context about the current repository and working directory.
Summarize recent changes and any relevant project structure.
```

Sidekick triggers

Each entry in `triggers` is either a bare event-name string, which fires an unlimited number of times, or an object with `event` and an optional `limit`.

Event	Description
`user.message`	Fires on every user message.

`session.context_changed`	Fires when the working directory, repository, or branch changes (for example, after `cd` or switching Git branches).

Trigger field	Type	Default	Description
`event`	`string`	Required	Session event type that launches this agent.
`limit`	`number`	Unlimited	Maximum number of times this trigger may fire per session. Must be a positive integer when set.

Sidekick configuration fields

Field	Type	Default	Description
`triggers`	`string[]` or object[]	Required	Session event types that launch this agent. At least one trigger is required.
`behavior`	`string`	`"restart"`	`"restart"`: cancel any prior run and start fresh on each trigger. `"persistent"`: keep the same long-lived run alive and deliver new messages into the existing loop instead of relaunching.
`maxSendsPerTurn`	`number`	`1`	Maximum inbox sends allowed per trigger. In `"persistent"` mode, each delivered user message resets this budget.

The `"restart"` behavior is suited to stateless context-gathering agents. The `"persistent"` behavior is suited to agents that accumulate state across turns.

Permission approval responses

When the CLI prompts for permission to execute an operation, you can respond with the following keys.

Key	Effect
`y`	Allow this specific request once.
`n`	Deny this specific request once.
`!`	Allow all similar requests for the rest of the session.
`#`	Deny all similar requests for the rest of the session.
`?`	Show detailed information about the request.

When the full dialog is shown, you can also choose from these options:

Option	Scope	Persistence
Once	Single use	None
This location	Until manually cleared	Saved to disk per location
Always	Permanent	Config file

The **This location** option appears when the CLI can determine a location key (Git root or current directory). It persists the approval to disk so the same permission is automatically granted the next time you work in that directory without prompting again.

When a shell command needs tool approval and access to paths outside the allowed directories, the CLI can combine both approvals in one prompt. Approving once grants path access for that run only. Approving for the session or location also grants path access for the rest of the session.

Use `/reset-allowed-tools` to clear session approvals for the current session.

Parent-domain URL approvals

URL approval prompts can offer a parent-domain scope, such as `https://*.example.com`, in addition to the exact origin. This approves every subdomain of the registrable domain, such as `api.example.com` and `flow.example.com`, for the rest of the session.

The scope has these restrictions:

- Approval applies only to the current session and is never saved to settings.
- The scheme and any non-default port must match.
- Hosts under a separately owned suffix, such as `s3.amazonaws.com`, are not covered by approving `*.amazonaws.com`. This keeps an approval for *.amazonaws.com from extending to buckets under s3.amazonaws.com, which may belong to other tenants.

- IP addresses, single-label hosts such as `localhost`, and public suffixes such as `github.io` do not get this option.

Security

Plan mode

`/plan` starts a planning session where Copilot CLI can explore and analyze your codebase but is blocked from editing your project files. Specifically:

- **Your project files are protected.** Any attempt to edit, patch, or run a shell command that would change a file in your workspace is blocked automatically—this isn't just a suggestion to the model, it's enforced directly, so it doesn't depend on the model choosing to behave.
- **The plan itself can still be written.** Copilot needs somewhere to keep notes and draft the plan, so it's allowed to create and edit files in its own private planning workspace (including the plan file you'll review and approve).
- **Delegated sub-tasks are protected too.** If Copilot spins up a helper session to research part of your question, that helper inherits the same restrictions and can't edit your project either.
- **It's a safety net, not a guarantee.** This protection is designed to catch clear, direct attempts to change your files—it's not an airtight lockdown. Some things are intentionally allowed through so research isn't overly restricted: for example, shell commands whose effect can't be clearly determined in advance, and calls to external/MCP tools you've connected. In practice this is rarely an issue, but plan mode should be thought of as "changes require your review before applying," not an absolute guarantee that nothing on disk can change.

Once you're happy with the plan, approve it to exit plan mode and let Copilot make the actual changes.

Command safety analysis

Shell commands are analyzed before execution to identify potentially dangerous patterns:

- File deletion (`rm -rf`)
- System modifications (`sudo` , `chmod 777`)
- Network exfiltration (`curl` with sensitive paths)
- Credential access (reading `.env` , SSH keys)
- Inline environment variable assignments that override dangerous variables (for example, `PATH=...` , `LD_PRELOAD=...`)

High-risk commands can trigger additional warnings and approval prompts. Whether a request needs your approval depends on the permissions in effect.

When sandboxing is enabled and the **Allow sandbox bypass** setting is on (the default), a synchronous shell command that's blocked by the sandbox's filesystem or network policy prompts you to re-run it outside the sandbox—no model round-trip is required. Approving the prompt re-runs the command and returns its output. Declining keeps the sandboxed (blocked) result.

A sandbox bypass prompt also offers a **Yes, and disable the sandbox for the rest of this session** option. Choosing it turns off sandboxing for every remaining command in the current session, not just the command being approved. This only applies to the current session—starting a new session with `/new` sandboxes commands again, and the option has no effect on the saved `sandbox` configuration setting or on other running sessions.

A detached (backgrounded) command can't be sandboxed at all, since the sandbox can't wrap a process that outlives the tool call. When the **Allow sandbox bypass** setting is on, this prompts for approval before the command starts, instead of being refused outright. Approving runs the command unsandboxed from the start.

On Windows hosts whose sandbox policy supports denial capture, a blocked interactive shell command escalates in a single step instead of jumping straight to a full bypass. Approving re-runs the command with file and process restrictions set to record instead of block (the network policy still applies), then falls back to the disclosed full bypass only if the command is still blocked. Only one prompt is shown for the entire escalation. A denied loopback or local-network socket (for example, binding or connecting to `127.0.0.1`) also reaches this same escalation instead of failing silently. The retry result is labeled "sandbox relaxed" when network policy is still enforced, or "sandbox bypassed" when fully unsandboxed. If the retry still fails, the message explains that host-level permissions still apply.

A command that fails with an `EPERM` or `EACCES` diagnostic naming a sandbox-blocked path now offers to run outside the sandbox, even when the command line itself never named that path.

When a denial is caused by a symlink whose target escapes what the sandbox policy grants, the denial explains that the granted path's link points outside the grant and names the real target, so you know exactly which path to add to your sandbox policy.

For more information, see Configuring local sandbox settings.

Environment variable denylist

The CLI blocks inline assignment of environment variables that can be exploited to execute arbitrary code even in otherwise read-only commands. Blocked categories include:

Category	Examples
Dynamic-linker injection	`LD_*`, `DYLD_*` (all prefixes)
Git indexed config overrides	`GIT_CONFIG_COUNT`, `GIT_CONFIG_KEY_*`, `GIT_CONFIG_VALUE_*` (all `GIT_CONFIG_` prefixes)
Git external program hooks	`GIT_EXTERNAL_DIFF`, `GIT_PROXY_COMMAND`

Git config file overrides	`GIT_CONFIG`, `GIT_CONFIG_GLOBAL`, `GIT_CONFIG_SYSTEM`
Shell PATH and startup files	`PATH`, `BASH_ENV`, `ENV`
Existing blocked vars	`PAGER`, `GIT_PAGER`, `GIT_EDITOR`, `VISUAL`, `EDITOR`, `GIT_SSH`, `GIT_SSH_COMMAND`, `GIT_ASKPASS`, `BROWSER`, `GH_BROWSER`

`web_fetch` SSRF protection

The `web_fetch` tool enforces server-side request forgery (SSRF) protections before making any HTTP request:

- **Protocol allowlist**: Only `http://` and `https://` URLs are permitted. `file://` and other schemes are rejected.
- **IP blocklist**: Requests to loopback addresses (`127.x.x.x`, `::1`), RFC-1918 private ranges (`10.x`, `172.16–31.x`, `192.168.x`), and cloud metadata endpoints (for example, `169.254.169.254`) are blocked by IP-literal check and DNS pre-resolution.
- **Validated redirects**: `web_fetch` follows `3xx` redirects (up to 10 hops, within a 60-second network budget), re-validating each hop's target against the same IP blocklist before following it. A redirect to a different origin than the original URL requires permission approval, the same as any other cross-origin fetch; same-origin redirects are followed without an extra prompt. The final result notes when content was `(redirected from <original URL>)`.

When sandboxing is enabled, requests are checked against the sandbox network policy (outbound access, local and private hosts) before the permission prompt, so a target blocked by policy fails fast instead of prompting. If the **Allow sandbox bypass** setting is on and the sandbox network proxy can't reach a URL—as opposed to the URL being denied by policy—`web_fetch` prompts you to retry the request outside the sandbox. Approving makes one more attempt at the fetch and returns its outcome.

To allow `web_fetch` to reach `localhost` during development—for example, for a local docs server—set the following environment variable:

```
export COPILOT_WEB_FETCH_ALLOW_LOCALHOST=1
```

Sandbox tool directory grants

When local sandboxing is enabled, Copilot CLI discovers the tool directories a sandboxed command is likely to need and grants each one **read-only** access, so a command can run an installed toolchain without being able to modify it. Discovery runs for each command, before the process starts, and reads two kinds of source from the command's environment.

- **`PATH`** (`Path` on Windows). Every directory listed is a candidate grant.

- **Named toolchain variables.** The CLI inspects the variables in the table below. A directory value grants that directory. A path list is split on the operating system's path separator (a semicolon on Windows, a colon elsewhere), and each entry becomes a candidate grant. The Git file variables are inspected only on macOS and Linux.

A directory candidate is granted only when it is an absolute path that exists as a directory. Relative paths, missing paths, filesystem roots (such as `/` or `C:\`), and system-critical locations are excluded from discovery. Examples of excluded locations are `%WINDIR%` on Windows and `/bin`, `/sbin`, `/usr/bin`, `/usr/sbin`, `/boot`, `/proc`, `/sys`, and `/dev` on macOS and Linux. The reason is logged to `sandbox_spawn`.

These grants keep their path spelling rather than automatically adding a symbolic link's target. If access through a link is blocked, grant the intended target explicitly. Duplicate directories are removed, case-insensitively on Windows.

Variable	Toolchain	Value	Typically set on
`BAZEL_VC`	Bazel / Visual C++	Single directory	Windows
`BAZEL_VS`	Bazel / Visual Studio	Single directory	Windows
`BAZELISK_HOME`	Bazelisk	Single directory	All
`BAZELISK_HOME_DARWIN`	Bazelisk	Single directory	macOS
`BAZELISK_HOME_LINUX`	Bazelisk	Single directory	Linux
`BAZELISK_HOME_WINDOWS`	Bazelisk	Single directory	Windows
`CONDA_PREFIX`	Conda	Single directory	All
`DOTNET_ROOT`	.NET	Single directory	All
`GIT_CONFIG_SYSTEM`	Git system configuration	File	macOS and Linux
`GIT_EXEC_PATH`	Git helpers	Single directory	All
`GIT_SSL_CAINFO`	Git certificate bundle	File	macOS and Linux
`GIT_SSL_CAPATH`	Git certificates	Single directory	All
`GIT_TEMPLATE_DIR`	Git templates	Single directory	All
`GOPATH`	Go	Path list	All
`GOROOT`	Go	Single directory	All
`JAVA_HOME`	Java	Single directory	All
`LD_LIBRARY_PATH`	Shared libraries	Path list	Linux

`NODE_PATH`	Node.js	Path list	All
`NVM_HOME`	Node.js (nvm)	Single directory	Windows
`NVM_SYMLINK`	Node.js (nvm)	Single directory	Windows
`PATH` / `Path`	Executables (all)	Path list	All
`PSModulePath`	PowerShell	Path list	All
`PYENV_ROOT`	Python (pyenv)	Single directory	All
`PYTHONHOME`	Python	Single directory	All
`PYTHONPATH`	Python	Path list	All
`RUSTUP_HOME`	Rust (rustup)	Single directory	All
`VCINSTALLDIR`	Visual C++	Single directory	Windows
`VCPKG_ROOT`	vcpkg	Single directory	All
`VIRTUAL_ENV`	Python (venv)	Single directory	All
`VSINSTALLDIR`	Visual Studio	Single directory	Windows

Except for the two Git file variables, each variable is read on every platform. The **Typically set on** column shows where each is normally populated. An unset variable contributes nothing.

`CARGO_HOME` is handled separately through tool-specific grants for selected Cargo directories and files. It does not grant access to the whole Cargo home. Developer-tool grants can also follow relocated caches, including Go and NuGet caches.

These are not the only automatic grants. With **Allow dev tool access** enabled, Copilot CLI grants selected user-profile application directories, including `~/.local/bin` and `~/.local/lib` on macOS and Linux and the immediate subdirectories of `%LOCALAPPDATA%\Programs` on Windows. It also grants tool-specific caches and registries, some of which are writable, such as npm's cache and shared build caches.

Run `/sandbox policy` to see the resolved paths grouped by source and access. The **Dev tools** section lists tools detected in the current directory and their paths. Add a command, such as `/sandbox policy npm install`, to inspect its developer-tool access without running it. The report does not create missing caches. For the concepts behind how the policy is assembled, see Understanding filesystem policies for local sandboxing in GitHub Copilot CLI.

OpenTelemetry monitoring

Copilot CLI and other Copilot clients support OpenTelemetry (OTel) monitoring. For configuration options, exported signals, and attributes, see OpenTelemetry reference.

OTel is off by default with zero overhead. It activates when any of the following conditions are met:

- `COPILOT_OTEL_ENABLED=true`
- `OTEL_EXPORTER_OTLP_ENDPOINT` is set
- `COPILOT_OTEL_FILE_EXPORTER_PATH` is set

Further reading

- GitHub Copilot CLI
- GitHub Copilot hooks reference
- GitHub Copilot CLI plugin reference
- GitHub Copilot CLI programmatic reference
- GitHub Copilot CLI configuration directory

Back to top

Was this Doc helpful?

Yes No

Help us make GitHub Docs great!

All Docs are open source. See something that's wrong or unclear? Submit a pull request.

Make a contribution

STILL NEED HELP?

Ask the GitHub community
Contact support
Expert services
Blog

GITHUB INC. © 2026

Terms | Privacy | Status | Pricing